EX.NO: 1	IMPLEMENT SYMMETRIC KEY ALGORITHMS
DATE:	

Aim:

To implement symmetric key algorithms for secure data encryption and decryption.

Algorithm:

For Encryption

1. Initialize the round keys using the main symmetric key.
2. Break the plain text into blocks, padding the last block if necessary.
3. For each block: a. Perform the initial round key addition. b. Perform multiple rounds (10,12, or 14 rounds based on key length):
   - Byte substitution using a substitution box(S-box).
   - Rowshifting within the block.
   - Columnmixing within the block.
   - Roundkey addition using the current roundkey. c. Perform the final round without the column mixing step.
4. Combine the encrypted blocks to create the ciphertext.

For Decryption

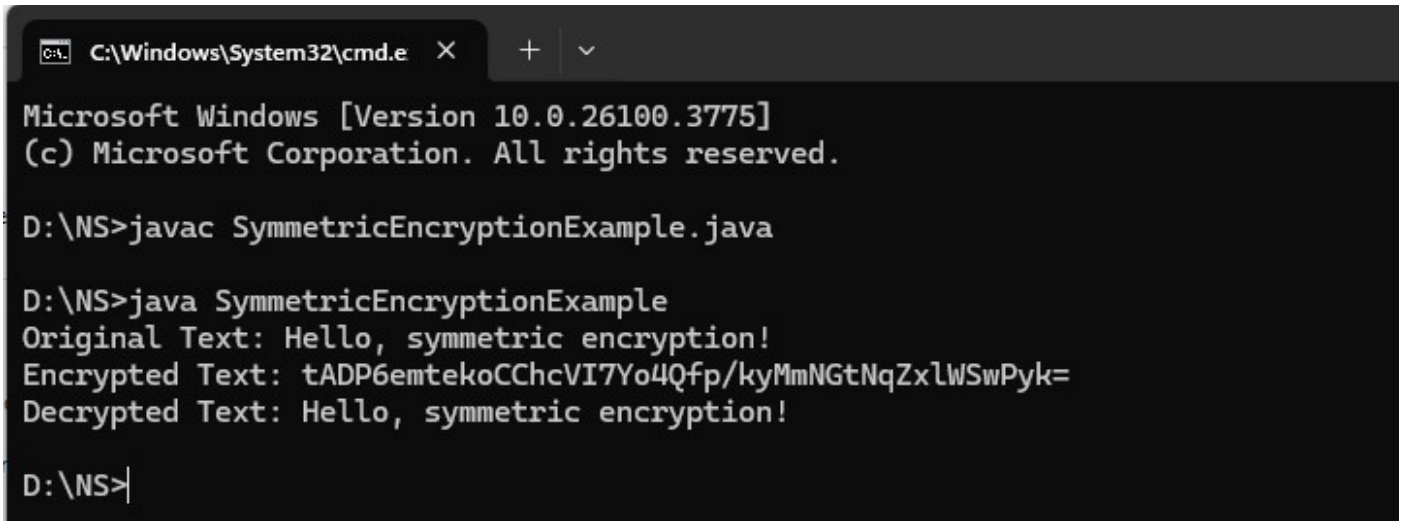
1. Initialize the round keys using the main symmetric key.
2. Break the cipher text into blocks.
3. For each block: a. Perform the initial round key addition. b. Perform the reverse of the encryption rounds in the reverse order:
   - Inverse byte substitution using an inverse S-box.
   - Inverse row shifting within the block.
   - Inverse round key addition using the current round key.
   - Inverse column mixing within the block (if not in the final round).

   c. Perform the final round without the inverse column mixing step.
4. Combine the decrypted block store cover the original plaintext.
5. Remove any padding added during encryption to obtain the actual original plaintext.

Program:

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
public class SymmetricEncryptionExample {
   public static void main(String[] args) throws Exception
      { String plaintext = "Hello, symmetric encryption!";
      System.out.println("Original Text: " + plaintext);
      // Generate a secret AES key
      KeyGenerator keyGen = KeyGenerator.getInstance("AES");
      keyGen.init(128); // AES key size (can also be 192 or 256 if supported)
      SecretKey secretKey = keyGen.generateKey();
      // Initialize cipher for encryption
      Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
      cipher.init(Cipher.ENCRYPT_MODE, secretKey);
      byte[] encryptedBytes = cipher.doFinal(plaintext.getBytes(StandardCharsets.UTF_8));
      String encryptedText = Base64.getEncoder().encodeToString(encryptedBytes);
      System.out.println("Encrypted Text: " + encryptedText);
      // Initialize cipher for decryption
      cipher.init(Cipher.DECRYPT_MODE, secretKey);
      byte[] decryptedBytes =
      cipher.doFinal(encryptedBytes);
      String decryptedText = new String(decryptedBytes, StandardCharsets.UTF_8);
      System.out.println("Decrypted Text: " + decryptedText);
   }
}
```

Output:



Result:

Thus, the java program to implement symmetric key algorithm is verified successfully and output is verified.

EX.NO: 2	IMPLEMENT ASYMMETRIC KEY ALGORITHMS AND KEY EXCHANGE ALGORITHMS
DATE:	

Aim:

To implement asymmetric key algorithms and key exchange algorithms using Java.

Algorithm:

Asymmetric Key Algorithms (RSA):

Key Generation:

1. Generate a pair of mathematically related keys: public key (for encryption) and private key (for decryption).
2. Select two large prime numbers, `p` and `q`.
3. Calculate the modulus `n = p * q`.
4. Calculate the totient `phi(n) = (p - 1) * (q - 1)`.
5. Choose an integer `e` (the public exponent) such that `1 < e <phi(n)` and `gcd(e, phi(n)) =1`.
6. Calculate the modular multiplicative inverse `d` of `e` modulo `phi(n)` (`d * e ≡ 1 (modphi(n))`).

Encryption:

1. Obtain the recipient's public key `(e, n)`.
2. Break the plaintext into smaller blocks.
3. For each block, calculate the cipher text `c = m^e mod n`, where `m` is the plaintext block.

Decryption:

Use the recipient's private key `d` to calculate the original message `m = c^d mod n`,

where `c` is the cipher text block.

Key Exchange Algorithms (Diffie-Hellman):

Key Generation:

1. Each party selects a large prime number `p` and a primitive root modulo `p`, denoted as `g`.

Key Exchange:

1. Both parties select their secret private keys, `a` and `b`, respectively.
2. They each calculate their public keys: `A=g ^a mod p `and `B=g ^b mod p`.
3. They exchange these public keys.

Using the received public keys, each party calculate the shared secret: `s= B ^a mod p` and `s = A ^b mod p`. Both parties now have the same shared secret `s`.

Output:

```
D:\NS>javac AsymmetricAndKeyExchangeExample.java

D:\NS>java AsymmetricAndKeyExchangeExample
Original Text: Hello, asymmetric encryption!
Encrypted Text: li+NXGCIGaWZSczpvgU/qirWdj5sYh5mcXnFH4BlSqy3XB6YG2CEvdrGqQUG4pWo9VoGTN1BNMav85x4anfkSWrYiVV28lZkReOTxAO4
/t6h+1XoYPVFA7hq+LjZPQfa5lbO5kQ6uqBzbdogx7qCb5S0Ajd4ez3Ryg0HSJjgtM7fbSNOGH/CoI9tfs6TJcPsm6DEfxeqbD5Q/L1THbJlfdflKjBAEjdS
YHM8M4Xe0BMC/yb629JcjEWwwLVUQs8Ufe3J4q0Md84MC6DwalYsL7vRbx/E05MzqBU0JpUz76ViPPq3Aom1eB/JFlHdxNkJ9hNLyEHZKX7jGsoYZMuQ8Q==
Decrypted Text: Hello, asymmetric encryption!
Shared Secret 1: bJuxesuEw8FkoLelbB3e0Gk5mmblhOIqcxZPI4wcPRBnC75e8Oeg2wLCaFuSA85rw/opBL8Oqmg64wCJJhCqh/eAfexxTXXDE26F/Fc
Yhzlu5u9Z53g97cvcey5Osb3NiqyHodhJIhuupke0OWDe/+eEuIcm29DjhMO674AzjLzvuak3sQxiCakgy8PVbIrarRzEmpZ+exny9h87zGNFgK0jlDULyZn
eYuEIaQQYHcv5/dAe8kcxyjA81FdgfL60NdeIqUSbMl9T2GA9P1QXb3wP+1vPxdJg+hpqElb30ypbphwhwUr8Okx1JPp1F5s8CWUinIUa5nbH197K64quMw=
=
Shared Secret 2: bJuxesuEw8FkoLelbB3e0Gk5mmblhOIqcxZPI4wcPRBnC75e8Oeg2wLCaFuSA85rw/opBL8Oqmg64wCJJhCqh/eAfexxTXXDE26F/Fc
Yhzlu5u9Z53g97cvcey5Osb3NiqyHodhJIhuupke0OWDe/+eEuIcm29DjhMO674AzjLzvuak3sQxiCakgy8PVbIrarRzEmpZ+exny9h87zGNFgK0jlDULyZn
eYuEIaQQYHcv5/dAe8kcxyjA81FdgfL60NdeIqUSbMl9T2GA9P1QXb3wP+1vPxdJg+hpqElb30ypbphwhwUr8Okx1JPp1F5s8CWUinIUa5nbH197K64quMw=
=
Shared secrets match: true

D:\NS>
```

Result:

Thus, the java program to asymmetric key algorithms and key exchange algorithms is executed successfully and output is verified.

EX.NO: 3	IMPLEMENT DIGITAL SIGNATURE SCHEMES
DATE:	

Aim:

To implement digital signature scheme using java program.

Algorithm:

Step 1: Key Pair Generation

Generate an RSA key pair with a specified key length.
Share the public key and private key with appropriate security measures.

Step 2: Creating a Digital Signature

Choose a message to be digitally signed.
Use the private key to generate a digital signature for the message.
Attach the digital signature to the message.

Step 3: Signature Verification

Receive a signed message and the associated digital signature.
Extract the public key from the sender.
Verify the authenticity of the message using the digital signature and the public key.

Step 4: Observation and Analysis

Observe the successful verification of authentic messages.
Attempt to verify messages with altered content or incorrect signatures to understand the verification process.

Step 5: Discussion and Reflection

Discuss the importance of digital signatures in ensuring data integrity and authentication.
Reflect on how the private key's security impacts the overall security of digital signatures.

Program:

```
import java.security.*;
import java.util.Base64;
import java.nio.charset.StandardCharsets;

public class DigitalSignatureExample {
   public static void main(String[] args) throws Exception {
      String message = "Hello, this is a message for digital signature!";
      System.out.println("Original Message: " + message);

      // Generate RSA key pair
      KeyPairGenerator keyPairGenerator = KeyPairGenerator.getInstance("RSA");
      keyPairGenerator.initialize(2048); // Key size
      KeyPair keyPair = keyPairGenerator.generateKeyPair();

      // Create a digital signature using the private key
      Signature signature = Signature.getInstance("SHA256withRSA");
      signature.initSign(keyPair.getPrivate());
      signature.update(message.getBytes(StandardCharsets.UTF_8));
```

```
        byte[] digitalSignature = signature.sign();
        System.out.println("Digital Signature: " + Base64.getEncoder().encodeToString(digitalSignature));

        // Verify the digital signature using the public key
        Signature verifier = Signature.getInstance("SHA256withRSA");
        verifier.initVerify(keyPair.getPublic());
        verifier.update(message.getBytes(StandardCharsets.UTF_8));
        boolean isVerified = verifier.verify(digitalSignature);

        // Output verification result
        if (isVerified) {
            System.out.println("Digital Signature Verified: The message is authentic.");
        } else {
            System.out.println("Digital Signature Verification Failed: The message may be tampered with.");
        }
    }
}
```

Output:

```
D:\NS>javac DigitalSignatureExample.java

D:\NS>java DigitalSignatureExample
Original Message: Hello, this is a message for digital signature!
Digital Signature: fV48stqnQ673Qo64LrcghydaatmkpLyjP6cyJQv3K+iT/RLP5SXDHp1XAgFjGTqeIk39vIwSBHG1ebPWyF1dF0a3mLKUyZ+J3adFE
qsDYVIMeyKAoIy9IQBp17pu/ftw7yHvy2R7uHUS4n1C9fgCMU0Uq77kcIdU7CvyVdkZLynHsAycvHmzdb1ER1u9mGvvRw65CcHMS7Vqbm+brUzQuA9DmKdS7
8FHaRRVeGEvJCpsyNWAvx6obrDfgSAEmCljHJEJCEqIkRPCp1DRXeWGhj7407u/JccUEGpLjn4VkAfpZjtXSMCgBUJkCsOrT40xBzs757jyYWSSopr15Dd6W
w==
Digital Signature Verified: The message is authentic.

D:\NS>
```

Result:

Thus, the java program to digital signature schemes is executed successfully and output is verified.

EX.NO: 4	INSTALLATION OF WIRESHARK, TCPDUMP
DATE:	

Aim:

To install Wireshark and tcpdump, observe data transferred in client-server communication using UDP/TCP protocols, and identify UDP/TCP datagrams for better understanding of network traffic analysis and protocol behaviour.

Procedure:

- Install Wireshark and tcpdump
  - Download and install Wireshark from the official website.
  - Install tcpdump using the package manager (e.g., apt-get, yum) on Linux systems.
- Set Up Client-Server Communication
  - Create a basic client-server communication using UDP or TCP (e.g., sending messages from client to server).
- Capture Network Traffic
  - Open Wireshark and choose the appropriate network interface.
  - Start capturing traffic to monitor the communication.
- Observe UDP Datagram
  - Filter captured packets to show only UDP packets.
  - Analyze source/destination ports, payload, and more.
- Observe TCP Segment
  - Filter captured packets to show only TCP packets.
  - Analyze ports, sequence/acknowledgment numbers, flags (SYN, ACK), and payload.
- Discussion and Analysis
  - Compare characteristics of UDP datagrams and TCP segments.
  - Discuss pros and cons of using UDP vs. TCP in different applications.
- Reflection
  - Reflect on the role of tools like Wireshark and tcpdump in understanding and troubleshooting network traffic.

Algorithm:

Step 1: Install Wireshark and tcpdump

Installing Wireshark:

Visit https://www.wireshark.org

Download and install for your OS (Windows, macOS, or Linux).

Installing tcpdump:

On Linux:

Open terminal

Use a package manager:

Debian/Ubuntu: sudo apt-get install tcpdump

CentOS/RHEL: sudo yum install tcpdump

Step 2: Observe Data Transferred in Client-Server Communication

Set Up a Simple Communication Scenario:

Choose client and server machines (on the same network).

Choose UDP or TCP for communication.

Write a simple client-server program or use tools like `nc` (TCP) or `ncat` (UDP).

Capturing Network Traffic:

Wireshark:

Open Wireshark

Select appropriate network interface

Click "Start" to begin capture

tcpdump:

Open terminal

Run the command:

```
sudo tcpdump -i <interface> -w capture.pcap
```

Step 3: Identify UDP/TCP Datagram

1. Stop the capture in Wireshark or tcpdump
2. In Wireshark:
    - Use filters:
        - `udp` → show only UDP packets
        - `tcp` → show only TCP packets
    - Click any packet to view details: source/destination ports, payload, etc.
3. In tcpdump:
    - Open `.pcap` file in Wireshark for graphical view
    - Use same filters as above

Identifying UDP Datagram:

- Look for packets using UDP protocol
- Observe source and destination ports, payload

Identifying TCP Segment:

- Look for packets using TCP
- Observe ports, sequence numbers, acknowledgment, flags (SYN, ACK), and payload.

Result:

Thus, the command for various activities is executed successfully and output is verified.

EX.NO: 5	CHECK MESSAGE INTEGRITY AND CONFIDENTIALITY USING SSL
DATE:	

Aim:

Write a Java program to check message integrity and confidentiality using ssl.

Algorithm:

Step 1: Key Pair Generation Generate an RSA key pair with a specified key length.

Share the public key and private key with appropriate security measures.

Step 2: Creating a integrity and confidentiality Choose a message to be data confidential.

Use the private key to generate a digital signature for the message. Attach the SSL to the message.

Step 3: Verification for SSL Receive a signed message and the associated SSL.

Extract the public key from the sender. Verify the integrity of the message using the SSL and the public key.

Step 4: Observation and Analysis Observe the successful verification of authentic messages.

Attempt to verify messages with altered content or incorrect SSl to understand the integrity and confidentiality process.

Step 5: Discussion and Reflection Discuss the importance of SSL in ensuring data integrity and confidentiality.

Reflect on how the private key's security impacts the overall security of SSL.

Create Environment variable for key tool.

Then **run this code** in class file location:

keytool -genkeypair -alias serverkey -keyalg RSA -keysize 2048 -validity 365 -keystore serverkeystore.jks -storepass changeit -keypass changeit -dname "CN=localhost, OU=Dev, O=MyOrg, L=City, S=State, C=US"

Program:

Server:

```
import javax.net.ssl.*;
import java.io.*;
import java.security.*;
import java.security.cert.Certificate;

public class SSLIntegrityServer {
   public static void main(String[] args) throws Exception {
      // Load keystore
      KeyStore keyStore = KeyStore.getInstance("JKS");
      keyStore.load(new FileInputStream("serverkeystore.jks"), "changeit".toCharArray());
      PrivateKey privateKey = (PrivateKey) keyStore.getKey("serverkey", "changeit".toCharArray());

      // Set system properties
      System.setProperty("javax.net.ssl.keyStore", "serverkeystore.jks");
```

```
        System.setProperty("javax.net.ssl.keyStorePassword",  "changeit");

        // SSL Server setup
        SSLServerSocketFactory ssf = (SSLServerSocketFactory) SSLServerSocketFactory.getDefault();
        SSLServerSocket serverSocket = (SSLServerSocket) ssf.createServerSocket(9999);
        System.out.println("Server ready. Waiting for client...");

        SSLSocket client = (SSLSocket) serverSocket.accept();
        System.out.println("Client connected.");

        String message = "This is a secure message.";
        Signature signature = Signature.getInstance("SHA256withRSA");
        signature.initSign(privateKey);
        signature.update(message.getBytes());
        byte[] sigBytes = signature.sign();

        ObjectOutputStream out = new
        ObjectOutputStream(client.getOutputStream()); out.writeUTF(message);
        out.writeInt(sigBytes.length);
        out.write(sigBytes);
        out.flush();

        System.out.println("Message and signature sent.");

        out.close();
        client.close();
        serverSocket.close();
    }
}
```

Client:

```
import javax.net.ssl.*;
import java.io.*;
import java.security.*;
import java.security.cert.Certificate;

public class SSLIntegrityClient {
    public static void main(String[] args) throws Exception {
        // Set truststore
        System.setProperty("javax.net.ssl.trustStore", "serverkeystore.jks");
        System.setProperty("javax.net.ssl.trustStorePassword",  "changeit");

        SSLSocketFactory sf = (SSLSocketFactory) SSLSocketFactory.getDefault();
        SSLSocket socket = (SSLSocket) sf.createSocket("localhost", 9999);

        ObjectInputStream in = new ObjectInputStream(socket.getInputStream());
        String message = in.readUTF();
        int sigLen = in.readInt();
        byte[] sigBytes = new byte[sigLen];
        in.readFully(sigBytes);

        System.out.println("Received message: " + message);

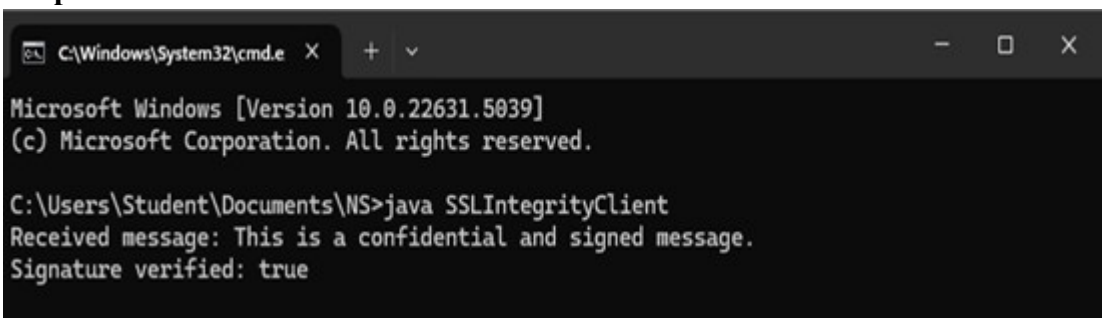
        // Load public key
        KeyStore keyStore = KeyStore.getInstance("JKS");
        keyStore.load(new FileInputStream("serverkeystore.jks"), "changeit".toCharArray());
        Certificate cert = keyStore.getCertificate("serverkey");
        PublicKey publicKey = cert.getPublicKey();
```

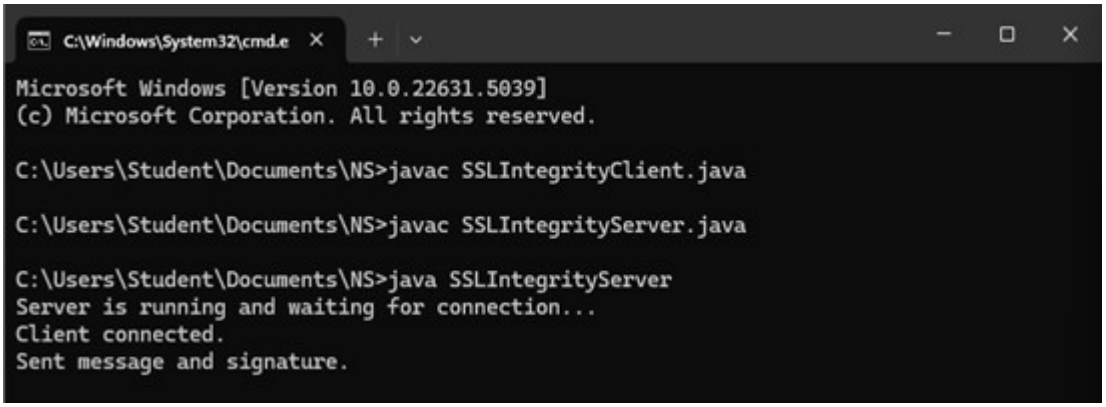
```
        Signature verifier = Signature.getInstance("SHA256withRSA");
        verifier.initVerify(publicKey);
        verifier.update(message.getBytes());

        boolean verified = verifier.verify(sigBytes);
        System.out.println("Signature verified: " + verified);

        in.close();
        socket.close();
    }
}
```

Output:





Result:

Thus the java program to check message integrity and confidentiality using ssl is executed successfully and output is verified.

EX.NO:6(a)	EAVESDROPPING
DATE:	

Aim:

To Implement Eaves Dropping Using Java Program.

Algorithm:

Create a Java Project: Start by creating a Java project in your favorite integrated development environment (IDE) or a simple text editor.

STEP 1: Import Required Libraries: You'll need libraries for hash functions and file handling. Import the necessary libraries.

STEP 2: Read the List of Hashed Passwords: Create a method to read the list of hashed passwords from a file. Assume the hashed passwords are stored in a file, one hash per line

STEP 3: Load the Dictionary: Create a method to load a dictionary file containing words or phrases you want to try as passwords

STEP 4: Perform the Dictionary Attack: Create a method to perform the dictionary attack

STEP 5: Main Method: In your main method, call the above methods to read hashed passwords, load the dictionary, and perform the dictionary attack

STEP 6: Prepare Input Files: Create two text files, one for the hashed passwords (hashed_passwords.txt) and one for the dictionary (dictionary.txt). Populate them with the appropriate data.

STEP 7: Run the Program: Run the Java program, and it will attempt to match the hashed passwords from your input file with the words or phrases in the dictionary file.

PROGRAM:

Server side:

```
// File: SecureServer.java

import java.io.*;

import java.net.*;


public class Server {

  public static void main(String[] args) {

    try {

      ServerSocket serverSocket = new ServerSocket(5000);

      System.out.println("Server listening on port 5000...");


      Socket socket = serverSocket.accept();
```

```
    System.out.println("Client connected");

    BufferedReader input = new BufferedReader(new InputStreamReader(socket.getInputStream()));

    String message;
    while ((message = input.readLine()) != null) {
      System.out.println("Received: " + message);
    }

    socket.close();
    serverSocket.close();
   } catch (IOException e) {
    e.printStackTrace();
   }
  }
}
```

Client side:

```
import java.io.*;
import java.net.*;

public class Client {
   public static void main(String[] args) {
       try {
           Socket socket = new Socket("127.0.0.1", 5000);

           PrintWriter output = new PrintWriter(
               socket.getOutputStream(), true);

           output.println("Hello from client");
           output.println("This is a secret message");
           output.println("password=12345");

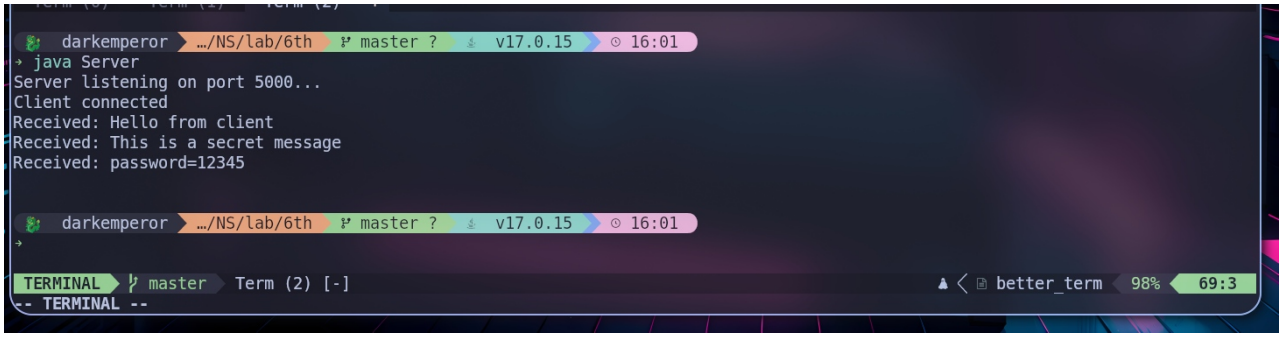


socket.close();
       } catch (IOException e) {
```

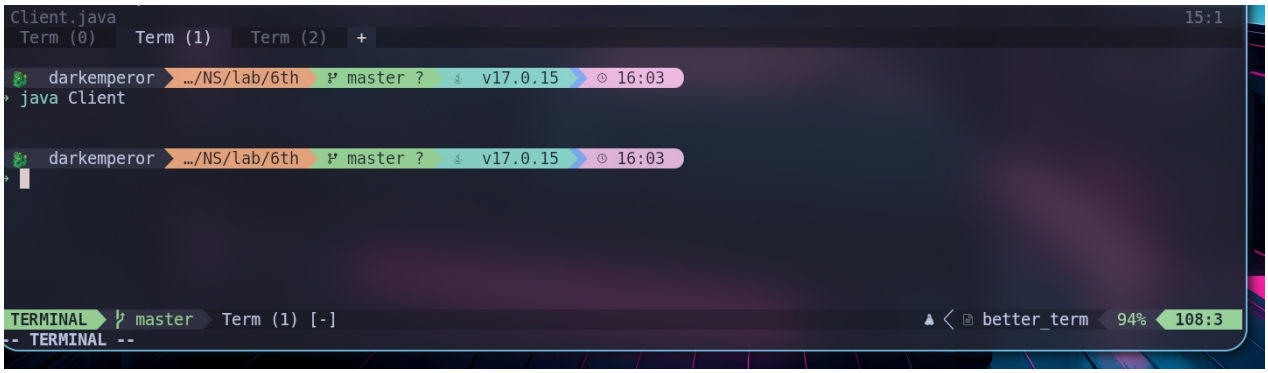
```
            e.printStackTrace();
        }
    }
}
```

Output:

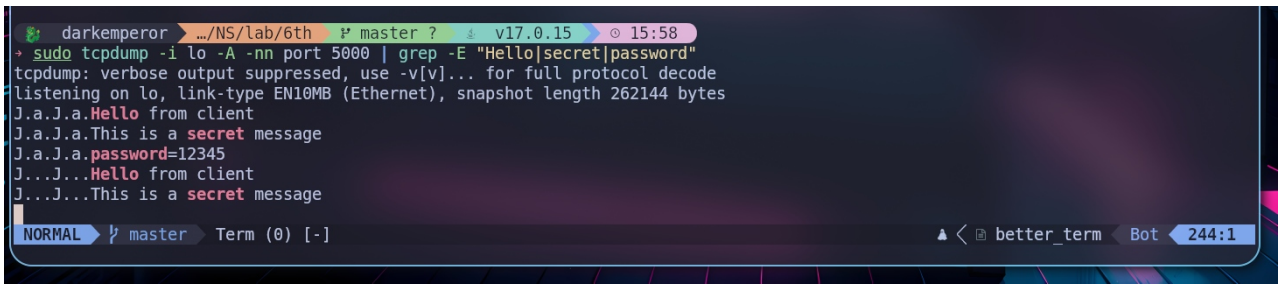
Server:



Client:



Tcpdump:



RESULT:

Thus the java program to implement Eavesdropping is executed successfully and output is verified.

EX.NO: 6(b)	DICTIONARY ATTACKS
DATE:	

Aim:

To Implement dictionary attacks Using Java Program.

Algorithm:

Create a Java Project: Start by creating a Java project in your favorite integrated development environment (IDE) or a simple text editor.

STEP 1: Import Required Libraries: You'll need libraries for hash functions and file handling. Import the necessary libraries.

STEP 2: Read the List of Hashed Passwords: Create a method to read the list of hashed passwords from a file. Assume the hashed passwords are stored in a file, one hash per line

STEP 3: Load the Dictionary: Create a method to load a dictionary file containing words or phrases you want to try as passwords

STEP 4: Perform the Dictionary Attack: Create a method to perform the dictionary attack

STEP 5: Main Method: In your main method, call the above methods to read hashed passwords, load the dictionary, and perform the dictionary attack

STEP 6: Prepare Input Files: Create two text files, one for the hashed passwords (hashed_passwords.txt) and one for the dictionary (dictionary.txt). Populate them with the appropriate data.

STEP 7: Run the Program: Run the Java program, and it will attempt to match the hashed passwords from your input file with the words or phrases in the dictionary file.

Program:

```
import java.io.*;
public class DictionaryAttack {
   public static void main(String[] args) {
      String username = "user123";
      String dictionaryFile = "passwords.txt";
      boolean accessGranted = false;
      try (BufferedReader reader = new BufferedReader(new FileReader(dictionaryFile)))
         { String password;
         while ((password = reader.readLine()) != null) {
```

```
            System.out.println("Trying password: " + password); // Debug output
            if (authenticate(username, password)) {
                System.out.println("Access granted for username: " + username + " with password: " + password);
                accessGranted = true;
                break;
            }
        }
    } catch (IOException e) {
        System.out.println("Error reading file: " + dictionaryFile);
        e.printStackTrace();
    }
    if (!accessGranted) {
        System.out.println("Dictionary attack failed. Access not granted.");
    }
  }
  private static boolean authenticate(String username, String password)
    { return password.equals("123456");
  }
}
```

Output:

```
D:\College\6th sem\NS>javac DictionaryAttack.java

D:\College\6th sem\NS>java DictionaryAttack
Trying password: password
Trying password: 111111
Trying password: qwerty
Trying password: abc123
Trying password: letmein
Trying password: 123456
Access granted for username: user123 with password: 123456

D:\College\6th sem\NS>
```

Result:

Thus the java program to implement Dictionary attack is executed successfully and output is verified.

EX.NO: 6(c)	MAN-IN-THE-MIDDLE ATTACK
DATE:	

Aim:

To guide Man In The Middle Attack and resolved it.

INTRODUCTION :

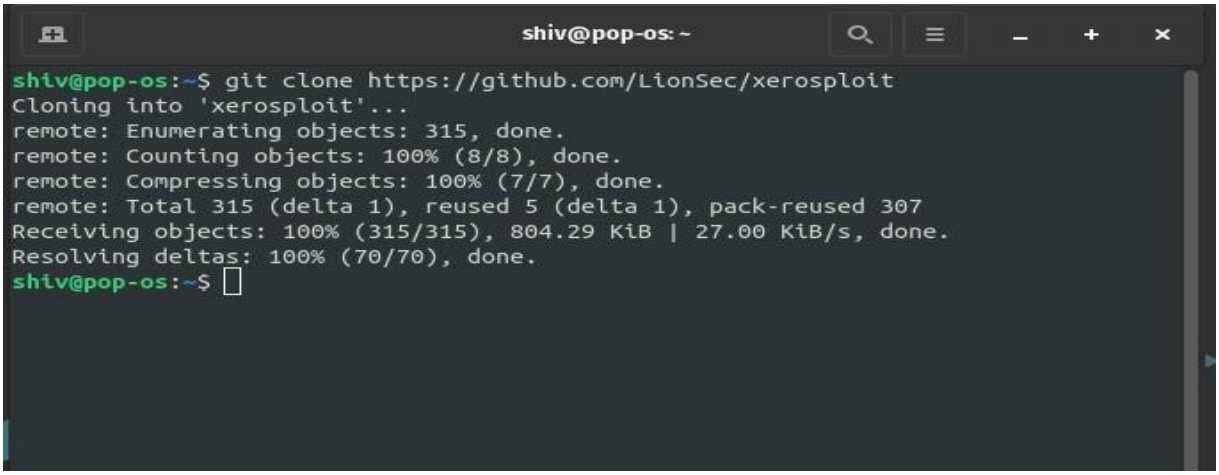
A man-in-the-middle (MITM) attack is a malicious technique in which an attacker intercepts and possibly alters the communication between two parties by positioning themselves between them.

By exploiting vulnerabilities in the communication channel, the attacker gains unauthorized access to sensitive information without the knowledge of the communicating parties.

Steps To Perform Advanced Man-in-the-Middle Attacks with Xerosploit:

Step 1: Clone the official Xerosploit repository from Github using the git clone command:

git clone https://github.com/LionSec/xerosploit



Step 2: Change the directory to xerosploit using the cd command. and then install the xerosploit:

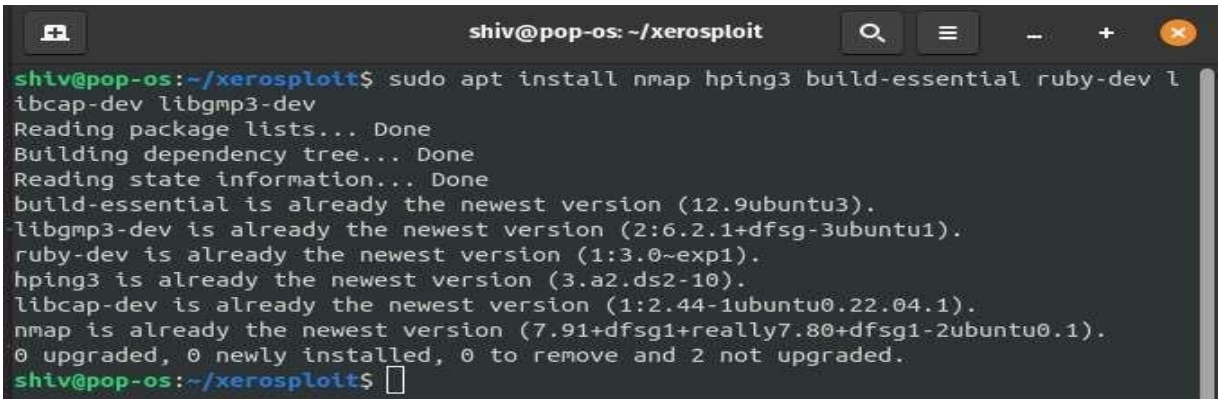
cd xerosploit

sudo python install.py



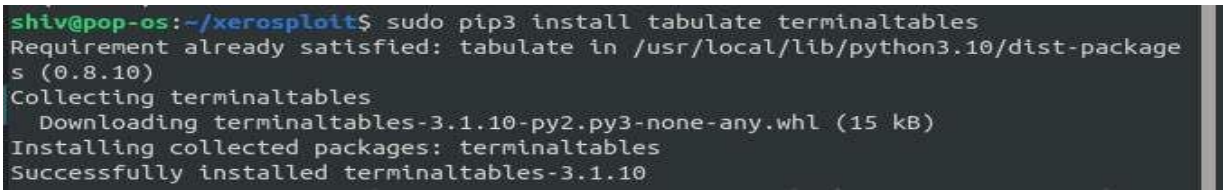
Step 3: Install it's all dependencies using the following command:

sudo apt install nmap hping3 build-essential ruby-devlibpcap-dev libgmp3-dev



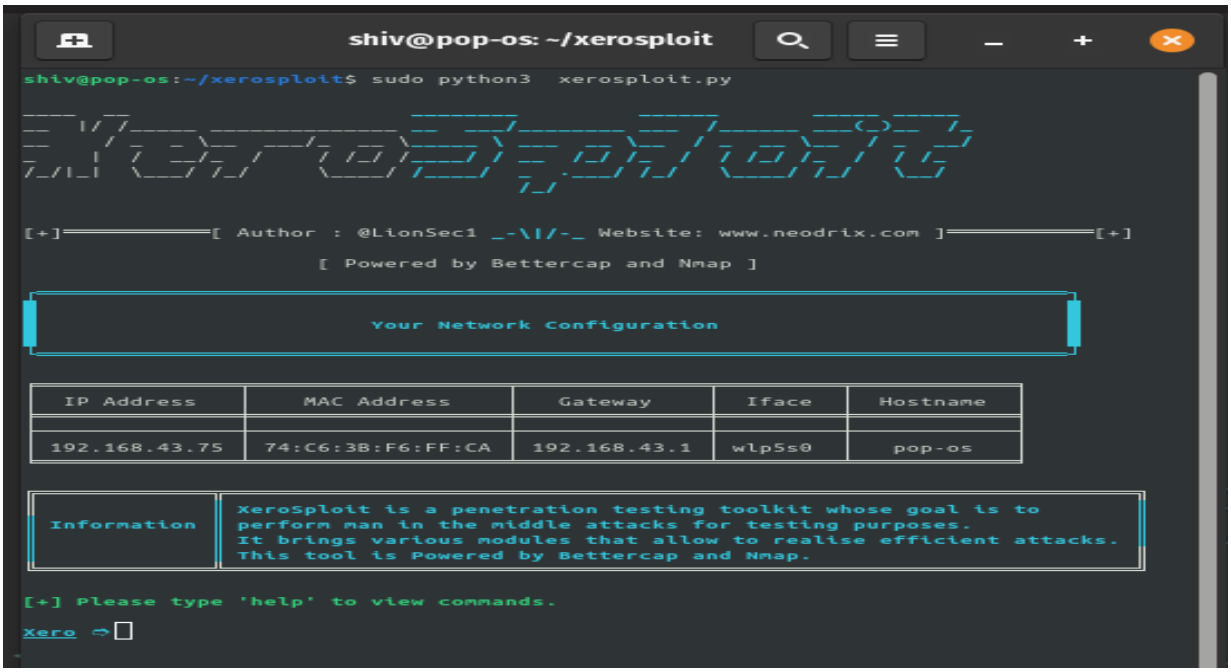
Step 4: Run the following command for setting up the terminaltables:

sudo pip3 tabulate terminaltables



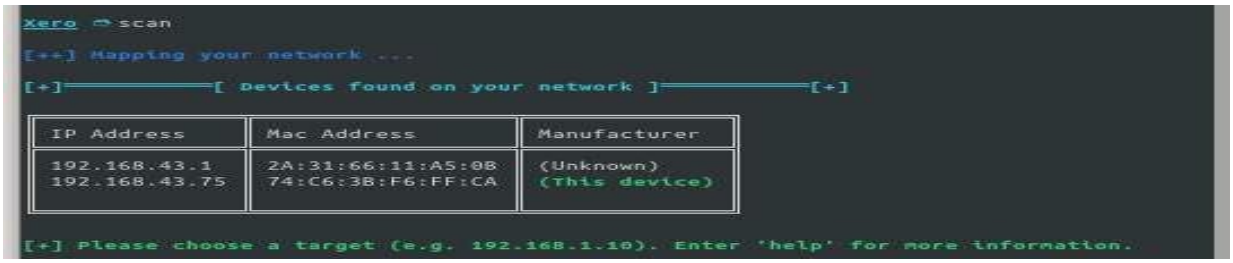
Step 5: Run Xerpsploit:

sudo python3 xerosploit.py



Step 6: Use the commands for various purposes like scanning the network use the below command.

Scan



Step 7: Select the target and run the module you want to execute.



Result:

Thus the man in the middle attacks discussed and all the modules are executed.

EX.NO: 7	SNIFF TRAFFIC USING ARP POISONING
DATE:	

Aim:

To guide Sniff Traffic Using Arp Poisoning and resolved it.

Enter the command:

ipconfig /all

You will get detailed information about all the network connections available on your computer. The results shown below are for a broadband modem to show the MAC address and IPv4 format and wireless network to show IPv6 format.

```
PS C:\Users\DivyaLakshmi> ipconfig /all

Windows IP Configuration

   Host Name . . . . . . . . . . . . : OG-Stark
   Primary Dns Suffix  . . . . . . . :
   Node Type . . . . . . . . . . . . : Mixed
   IP Routing Enabled. . . . . . . . : No
   WINS Proxy Enabled. . . . . . . . : No

Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Realtek PCIe GbE Family Controller
   Physical Address. . . . . . . . . : F8-0D-AC-C4-FC-32
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
```

What is ARP Poisoning?

ARP is the acronym for Address Resolution Protocol. It is used to convert IP address to physical addresses [MAC address] on a switch. The host sends an ARP broadcast on the network, and the recipient computer responds with its physical address [MAC Address]. The resolved IP/MAC address is then used to communicate. ARP poisoning is sending fake MAC addresses to the switch so that it can associate the fake MAC addresses with the IP address of a genuine computer on a network and hijack the traffic.

ARP Poisoning Countermeasures

Static ARP entries: these can be defined in the local ARP cache and the switch configured to ignore all auto ARP reply packets. The disadvantage of this method is, it's difficult to maintain on large networks. IP/MAC address mapping has to be distributed to all the computers on the network.

ARP poisoning detection software: these systems can be used to cross check the IP/MAC address resolution and certify them if they are authenticated. Uncertified IP/MAC address resolutions can then be blocked.

Operating System Security: this measure is dependent on the operating system been used.

The following are the basic techniques used by various operating systems.

- Linux based: these work by ignoring unsolicited ARP reply packets.
- Microsoft Windows: the ARP cache behaviour can be configured via the registry.

The following list includes some of the software that can be used to protect networks against sniffing;

- AntiARP– provides protection against both passive and active sniffing
- Agnitum Outpost Firewall–provides protection against passive sniffing

- XArp– provides protection against both passive and active sniffing
- Mac OS: ArpGuard can be used to provide protection. It protects against both active and passive

sniffing. Hacking Activity: Configure ARP entries in Windows

We are using Windows 7 for this exercise, but the commands should be able to work on other versions of windows as well.

Open the command prompt and enter the following command:

arp –a

HERE,

- aprcalls the ARP configure program located in Windows/System32 directory
- -a is the parameter to display to contents of the ARP

cache You will get results similar to the following:

```
C:\Windows\System32>arp -a

Interface: 192.168.97.202 --- 0xd
  Internet Address      Physical Address      Type
  192.168.97.38         10-68-38-f5-9d-e4     dynamic
  192.168.97.137        1a-e4-04-7a-7b-fa     dynamic
  192.168.97.158        d8-80-83-74-70-df     dynamic
  192.168.97.202        28-cd-c4-b1-43-1d     static
  192.168.97.203        18-cf-5e-f8-1c-71     dynamic
  192.168.97.255        ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.192.152.143       01-00-5e-40-98-8f     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

Note: dynamic entries are added and deleted automatically when using TCP/IP sessions with remote computers.

Static entries are added manually and are deleted when the computer is restarted, and the network interface card restarted or other activities that affect it.

Adding static entries

Open the command prompt then use the ipconfig /all command to get the IP and MAC address:

```
Wireless LAN adapter WiFi:

   Connection-specific DNS Suffix  . : lan
   Description . . . . . . . . . . . : Realtek RTL8822BE 802.11ac PCIe Adapter
   Physical Address. . . . . . . . . : 28-CD-C4-B1-43-1D
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
   IPv6 Address. . . . . . . . . . . : 2409:40f4:4048:ba9:a09d:1161:c392:4ead(Preferred)
   Temporary IPv6 Address. . . . . . : 2409:40f4:4048:ba9:a0d6:f86c:1e6c:3622(Preferred)
   Link-local IPv6 Address . . . . . : fe80::949d:e0e2:d0b5:2db4%13(Preferred)
   IPv4 Address. . . . . . . . . . . : 192.168.31.102(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Lease Obtained. . . . . . . . . . : 07 May 2025 18:54:55
   Lease Expires . . . . . . . . . . : 08 May 2025 02:55:04
   Default Gateway . . . . . . . . . : fe80::4e82:a9ff:feb9:6186%13
                                       192.168.31.1
   DHCP Server . . . . . . . . . . . : 192.168.31.1
   DHCPv6 IAID . . . . . . . . . . . : 170446276
   DHCPv6 Client DUID. . . . . . . . : 00-01-00-01-29-C7-F1-EC-F8-0D-AC-C4-FC-32
   DNS Servers . . . . . . . . . . . : 192.168.31.1
   NetBIOS over Tcpip. . . . . . . . : Enabled
```

The MAC address is represented using the Physical Address and the IP address is IPv4Address

Enter the following command:

arp –s IP address <space> Physical Address

```
C:\Windows\System32>arp -s 192.168.97.202 28-CD-C4-B1-43-1D

C:\Windows\System32>arp -a

Interface: 192.168.97.202 --- 0xd
  Internet Address      Physical Address      Type
  192.168.97.38         10-68-38-f5-9d-e4     dynamic
  192.168.97.137        1a-e4-04-7a-7b-fa     dynamic
  192.168.97.158        d8-80-83-74-70-df     dynamic
  192.168.97.202        28-cd-c4-b1-43-1d     static
  192.168.97.203        18-cf-5e-f8-1c-71     dynamic
  192.168.97.255        ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.192.152.143       01-00-5e-40-98-8f     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

Note: The IP and MAC address will be different from the ones used here. This is because they are unique.

Use the following command to view the ARP cache:

arp –a

You will get the following results:

```
C:\Windows\System32>arp -a

Interface: 192.168.97.202 --- 0xd
  Internet Address      Physical Address      Type
  192.168.97.38         10-68-38-f5-9d-e4     dynamic
  192.168.97.137        1a-e4-04-7a-7b-fa     dynamic
  192.168.97.158        d8-80-83-74-70-df     dynamic
  192.168.97.202        28-cd-c4-b1-43-1d     static
  192.168.97.203        18-cf-5e-f8-1c-71     dynamic
  192.168.97.255        ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.192.152.143       01-00-5e-40-98-8f     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

Note the IP address has been resolved to the MAC address we provided and it is of a static type.

Deleting an ARP cache entry

Use the following command to remove an entry

arp –d IP Adress

```
C:\Windows\System32>arp -d 192.168.97.202

C:\Windows\System32>
```

Result:

The expected output is achieved with the command of Sniff Traffic Using Arp Poisoning.

EX.NO: 8	INTRUSION DETECTION SYSTEM
DATE:	

Aim:

To demonstrate an Intrusion Detection System (IDS) using the Snort software tool.

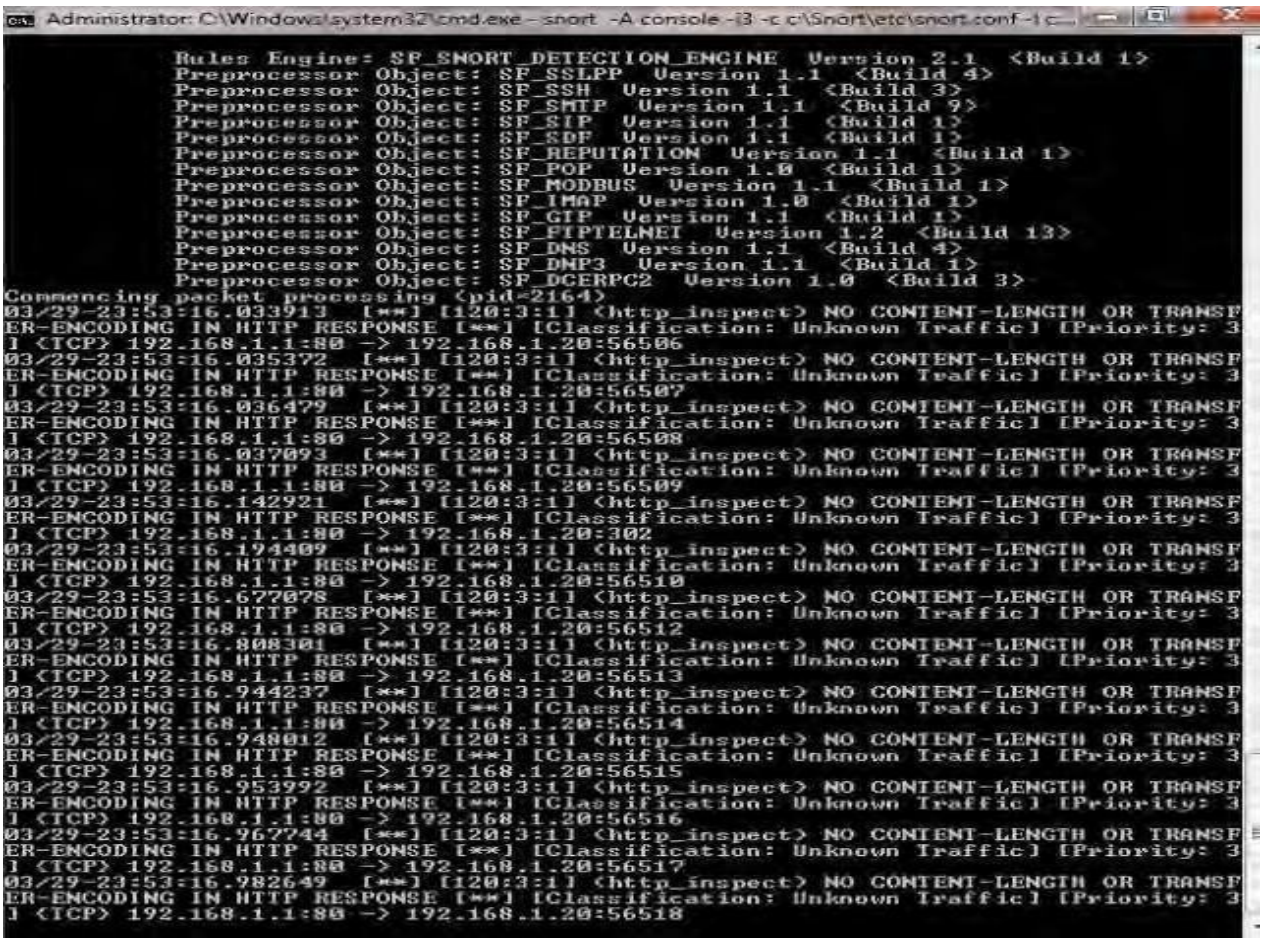
Steps For Configuring An Intrusion Detection System:

- **Download Snort:**
  Visit the official Snort website to download Snort:
  http://www.snort.org/snort-downloads
- **Download Snort Rules:**
  Go to https://www.snort.org/snortrules and register to download the latest Snort rule sets.
  (Note: It is recommended to download updated rules frequently.)
- **Install Snort:**
  Double-click the downloaded .exe file to install Snort. By default, Snort will be installed in the following directory:
  C:\Snort
  Important: Ensure that **WinPcap** is installed before running Snort. You can download it from:
  https://www.winpcap.org/install/
- **Extract the Rules File:**
  The rules file is typically in .gz format. Use **WinRAR** or any similar tool to extract it.
- **Copy Snort Rules:**
  From the extracted folder, copy all files from the rules directory and paste them into the following directory:
  C:\Snort\rules
- **Configure snort.conf File:**
  From the extracted folder, copy the snort.conf file located in the etc directory and paste it into:
  C:\Snort\etc
  (If prompted, overwrite the existing file.)
  Note: If you later download a new snort.conf file, you must modify it accordingly for Snort to function properly.
- **Open Command Prompt:**
  Launch cmd.exe, and navigate to the Snort binary directory by typing: cd \Snort\bin
- **Run Snort in Sniffer Mode:**
  Use the following command to start Snort in sniffer mode:
  snort -dev -i 3
  snort -W



- To run Snort in sniffer mode (for testing): snort -dev -i 3
  (Replace 3 with your actual interface number.)
- To run Snort in IDS mode, you need to configure snort.conf according to your network.
- In snort.conf, find and update this line:
  var HOME_NET 192.168.1.0/24
  (Change the IP range to match your network. You may see "any" as default.)
- If you have DNS servers on your network, add:
  var DNS_SERVERS [192.168.1.1,
  192.168.1.2]
  (Replace with your actual DNS IPs.)
- Set the rule path:
  var RULE_PATH c:\Snort\rules
- Update all library paths in snort.conf. Replace any of the following default Unix-style paths like:
  /usr/local/lib/...
  with C:
  \Snort\lib
  Also update the dynamic preprocessor path:
  dynamicpreprocessor directory C:\Snort\lib\snort_dynamicpreprocessor
- Set the dynamic engine path:
  dynamicengine C:\Snort\lib\snort_dynamicengine\sf_engine.dll
- Include the configuration files:
  include c:\Snort\etc\classification.config
  include c:\Snort\etc\reference.config
- Uncomment the ICMP rules line (remove # if it's commented): include $RULE_PATH/icmp.rules
- Uncomment this line as well (if needed):
  include $RULE_PATH/icmp-info.rules
- Add alert output to a file:
  output alert_fast: snort-alerts.ids
- Comment out whitelist and blacklist if not used:
  include $WHITE_LIST_PATH/white_list.rules
  include $BLACK_LIST_PATH/black_list.rules
- Comment out unused preprocessors (optional but recommended for simplicity):
  preprocessor normalize_ip4
  preprocessor normalize_tcp: ips
  preprocessor normalize_icmp4
  preprocessor normalize_ip6
  preprocessor normalize_icmp6
- Save and close the snort.conf file.
- To start Snort in IDS mode, use:
  snort -c c:\Snort\etc\snort.conf -l c:\Snort\log -i 3
  (Replace 3 with your interface number.)
- To generate log files in ASCII mode, use:
  snort -A console -i 3 -c c:\Snort\etc\snort.conf -l c:\Snort\log -K ascii
- From another machine, scan the Snort host using ping or Nmap (e.g., ZenMap).
  Check the log folder at C:\Snort\log
  You should see alert files and folders named with the IP address of the scanning machine if Snort is logging properly.

Snort monitoring traffic–



RESULT:

Thus, the Intrusion Detection System (IDS) has been successfully demonstrated using the open-source Snort Intrusion Detection Tool.

EX.NO: 9	NETWOTK MONITORING TOOLS
DATE:	

Aim:

To achieve the network monitoring tools with the required function and software.

Introduction:

Networks are the fundamentals behind businesses worldwide. It plays a pivotal role in serving your employees for administrative purposes and your clients across the continents. The networks help you keep information in a centralized location - accessible to those who need and restrict every other inbound request. So how do you provide continuous top-notch end user experience and maintain your rapidly evolving network? Only by monitoring the availability, health, and performance of your networks over time with the help of reliable, real-time network monitoring tools.

Requirements of a network monitoring tool:

While selecting a network monitortool for your IT environment, it is important to weigh in your current requirements and also your future needs because choosing the right one among various tools for network monitoring is crucial. Some of the essential elements that a network monitoring tool requires are:

- Real time monitoring
- Comprehensive monitoring capabilities
- scalability
- automation
- user management

Real time monitoring:

Learning that your network is down from your end users is the nightmare that every IT admin tries to avoid. The network monitoring application and its reporting tools must provide performance insights into your network in real time. This helps you identify performance hiccups early and avoid potential outages.

Comprehensive monitoring capabilities:

Employing individual network monitoring tools for monitoring various network components such as switches/routers, servers, virtual environment, HCI, applications, storage devices, etc., and also employing network monitoring tools for Windows and Linux environments separately is more trouble than help as the tools themselves require constant management, additional resources, and there is also a certain learning curve associated with each network monitoring tool. Therefore, the network performance monitoring tools should support extensive monitoring in a single console.

Scalability

Network scalability is an important aspect to be considered when selecting enterprise network monitoring tools. A network management tool or **network monitoring software** can be called as scalable when it is more adaptable to the changing needs or demands of the business or users. Scalability helps a network to stay in par with increased productivity, trends, changing needs and new adaptations. Scalabiity ensures that the overall network performance
does not significantly degrade, even if the size of the network is increasing. Businesses also need remote network monitoring tools for managing multiple geographical locations from one console.

Automation

In network monitoring, automation helps network performance monitoring tools to react based on threshold values or a set of framed rules/ criteria being met. With automation, the monitoring network tools can automatically detect and troubleshoot problems (proactive monitoring), send alert notifications, forecast storage growth, and much more. When you are monitoring and managing multiple devices in your environment, automation comes in super handy and saves you ample time and resources, making it an important usability aspect of your network monitoring solutions**.**

User Management

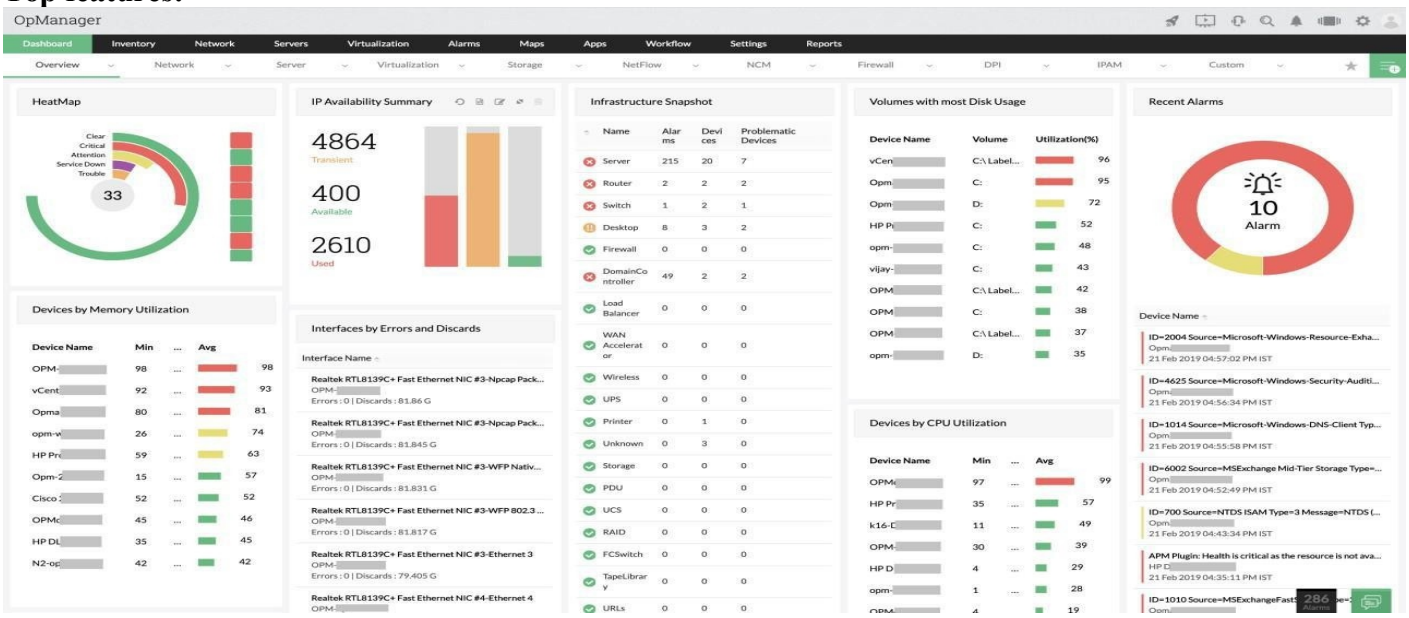
User Management helps organizations ensure network security by providing access to the designated users only. Apart from providing access to users with roles, the network monitoring tool should also define the scope for users. This helps IT teams with multiple staffs as it clearly defines their operational boundaries. Network monitoring tools with the above features are excepetionally benefiting to your business.

Why is ManageEngineOpManager considered one of the best network monitor tools in the market?

OpManager is a powerful network management and monitoring tool that monitors switches, routers, servers, WLC, load balancers, VPN, printers, firewalls, VMs, Nutanix environments, and anything that has an IP and connected to the network - in a single console.

- Top features
- Comprehensive monitoring capabilities
- Customer reviews

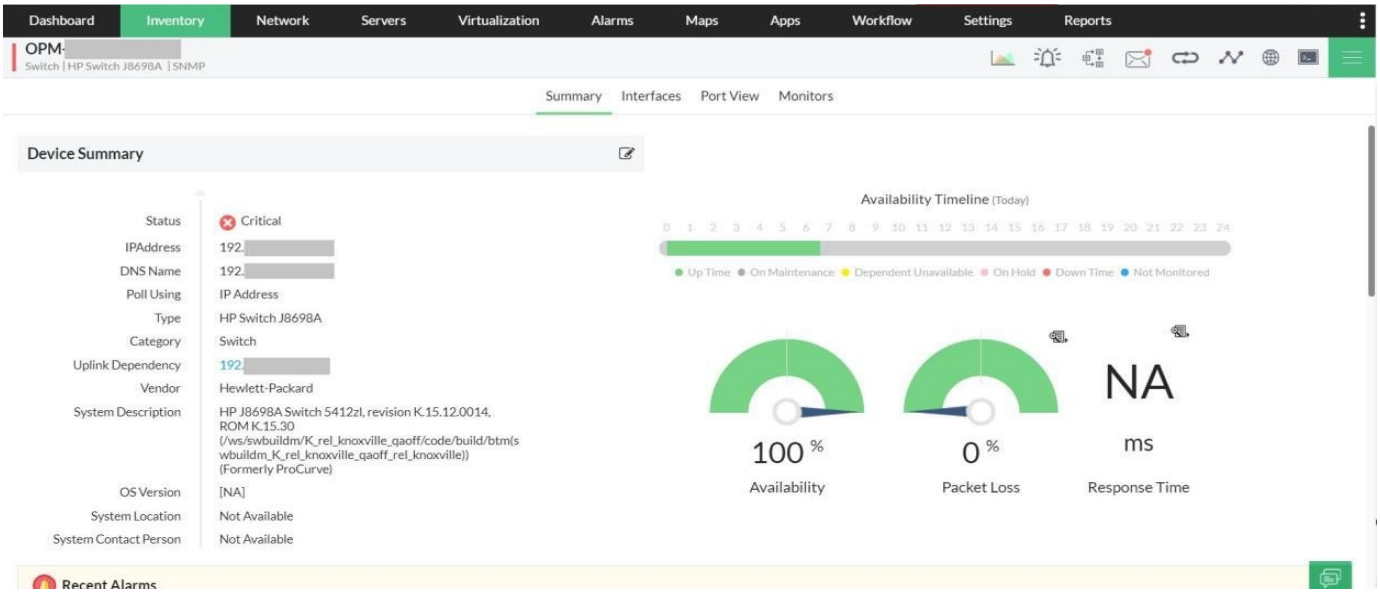
Top features:



Comprehensive monitoring capabilities

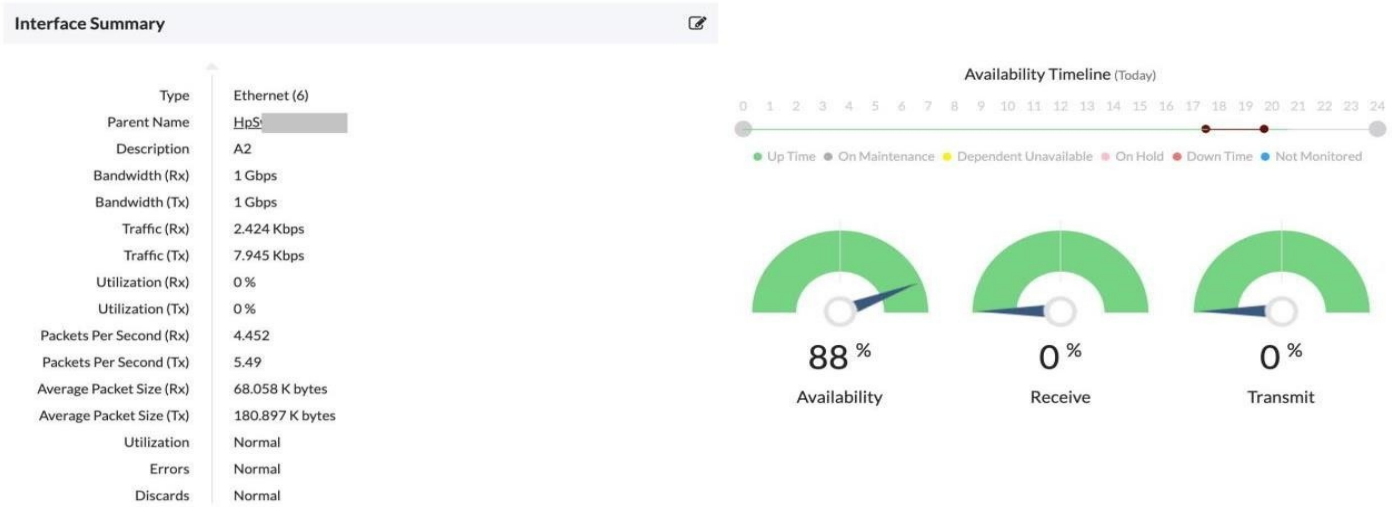
Switch/Router monitoring

Network switches and routers form the backbone of any IT infrastructure. Any issue with switch breaks the end user connectivity with the network. Using OpManager, you can monitor switches, and routers from the likes of Cisco, Juniper, Aruba, ZTE, and many other vendors for availability, health, and performance in real-time for 2,000+ parameters and avoid possible network pitfalls. Apart from monitoring switches, OpManager maps switch ports to devices and monitors the availability of the switch ports.



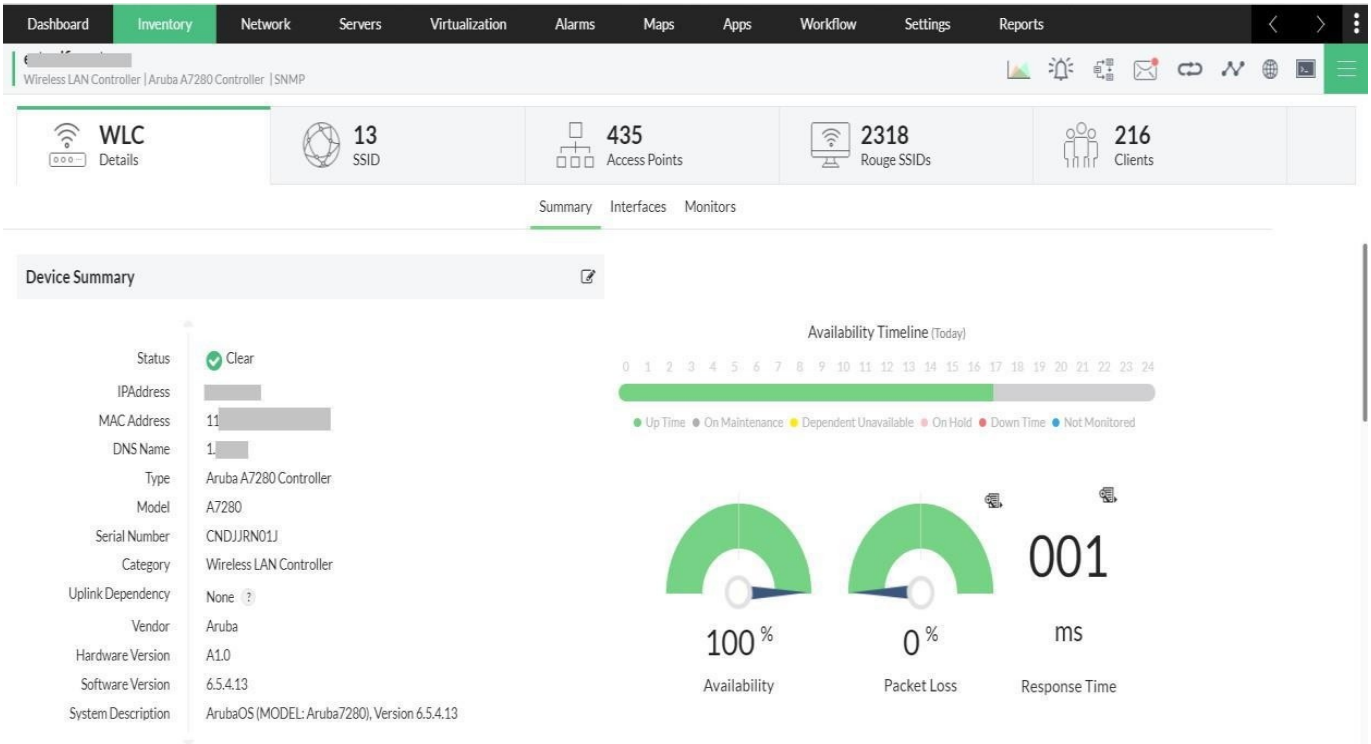
Network interface monitoring

Network interfaces are one of key performance indicators (KPI) as they help identify network performance degradation at the earliest. OpManager, the best network monitoring tool, monitors interfaces using SNMP and provides a single customizable dashboard to view and analyze bandwidth performance and network traffic for your IT network. You can monitor interfaces by checking the availability status of interfaces and monitor traffic speed on the interface, errors, discards, etc. using OpManager.



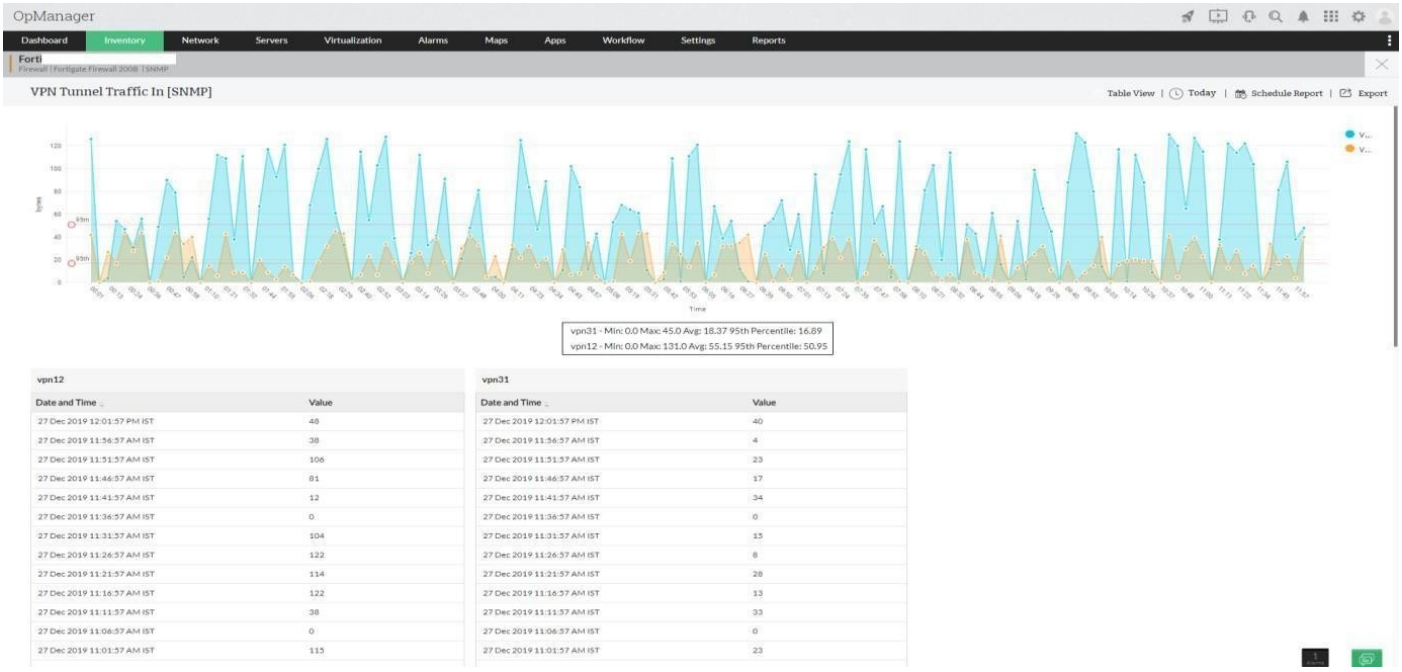
WLC monitoring

Op Manager's multi-vendor WLC monitoring module allows you to keep your network intact by providing in-depth visibility of your wireless LAN controller (WLC), its associated service set identifiers (SSIDs) and access points (APs). Cisco's WLC monitoring tool in OpManager allows direct discovery of Cisco WLC and their associated SSIDs, APs and helps you monitor the overall performance of your wireless network with the help of Cisco WLC monitor. The WLC snapshot page provides inventory information, the device availability status, and other similar information. In addition, knowing the top five access points based on usage tells you who the top talkers are in your WLC environment, and custom dials display information on various parameters, including CPU and memory usage.



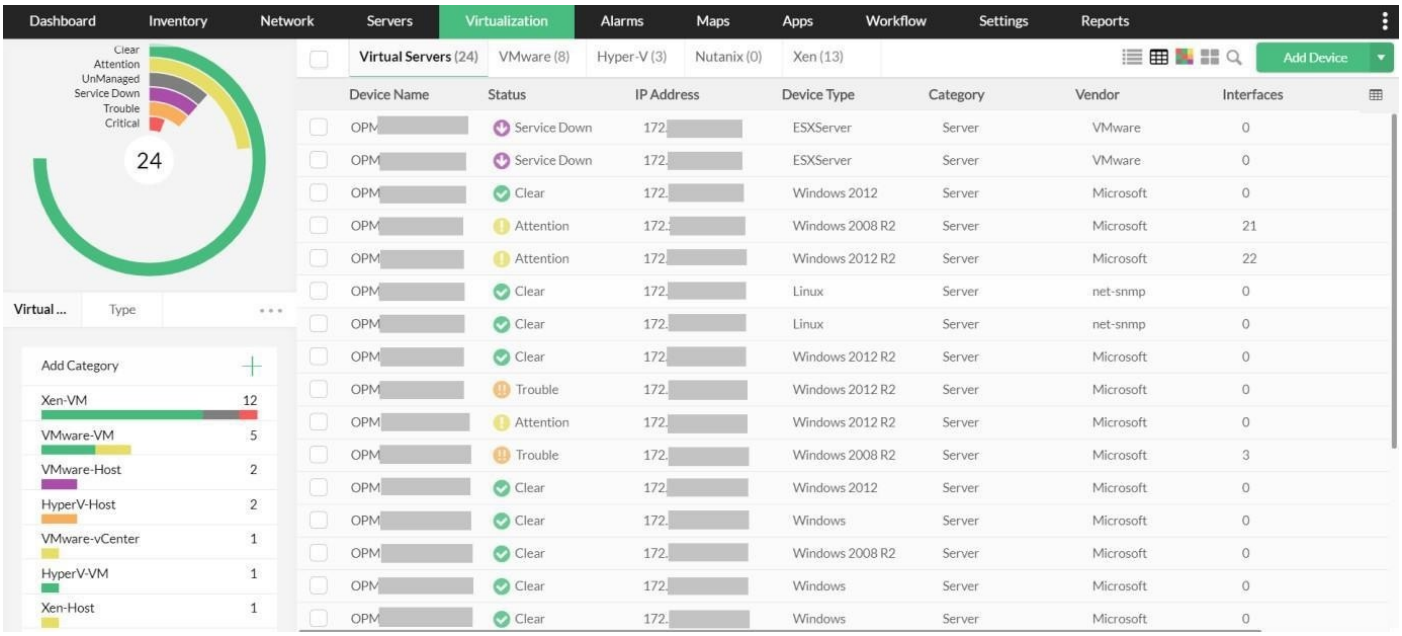
VPN monitoring

Organizations allow connections into their networks through VPNs for their remote workforce. These connections can sometimes be compromised, resulting in data theft or network attacks. With a monitoring tool like OpManager, you can monitor your VPN by tracking the number of active VPN sessions, VPN tunnel status, and VPN tunnels count in real time, and also receive instant alerts on VPN connection regularities making your network secure and keeping your remote productivity issues at bay.



Hybrid environment monitoring

Every organization's network has unique needs, so top-of-the-line networking technologies are employed to address them. This helps networks deliver business services but also poses a challenge with monitoring and managing the network. Using multiple network management tools is not efficient and cost effective. With OpManager, apart from monitoring switches, servers, etc., you can monitor VMware, Hyper-V, Hypervisors, Cisco UCS, Nutanix infrastructures, and more, all within a unified console, making it the best network monitor.
Additionally, you can monitor your WAN with Cisco IP SLA using OpManager.



Mobile application

Access your OpManager's network monitoring and reporting anytime and anywhere using the new ManageEngineOpManager mobile application. Available for both Android and iOS, this lOMoARcPSD|28361946 lets you visualize your infrastructure, act on the alerts, drill-down to the root cause of the problem without having to be physically present in your server room to resolve a fault.

OpManager makes your work easy

Apart from the above, OpManager, your comprehensive network monitoring solution monitors Windows servers, Linux servers, storage devices, Windows services, processes and scales upto 30,000 devices out of the box. This network software makes network monitoring effortless with intelligent automations, ML-based forecasting, and extensive protocol support.

Here are some of OpManager'snetwork monitoring applications:

1. **Storage capacity forecasting:** With the help of ML-based forecasting techniques, this network monitoring and reporting software pinpoints when the device storage will reach 80 percent, 90 percent, and 100 percent of the allocated storage, and helps with planning purchase decisions.

2. **Notification profiles:**OpManager lets you notify network faults via Slack channels, trouble tickets, emails, SMS, and web alarms if they are not acknowledged, so no alarm goes unnoticed.

3. **Alarm Escalation:** Alarm escalation rules can be configured for mission-critical devices such as application servers, so any fault pertaining to availability, health, and performance is escalated to a higher authority via email or SMS based on user-defined criteria.

4. **Support for multiple vendors:**OpManager offers support for more than 53,000 vendor templates, so you can efficiently manage your network devices from vendors such as Cisco, Juniper, Fortigate, and many more. These templates can also be customized to address your organization's unique needs.

5. **Support for wide range of protocols:**OpManager supports communication protocols such as ICMP, and LAN management protocols such as SNMP, WMI, CLI, and more.

6. **Discovery Rule Engine:** Discovery Rule Engine automatically associates device templates and rules to network devices as defined by the user, thereby automating routine tasks, and saving valuable time and resources.

7. **In-built troubleshooting tools:**OpManager offers multiple tools such as Ping, SNMP Ping, Proxy Ping, Traceroute, WMI Query Tool, CLI Query Tool, and more that aid in troubleshooting network issues within OpManager.

8. **Dashboards:**OpManager provides intuitive dashboards that provide a 360-degree view of your entire IT infrastructure on one screen, and makes fault identification easier.

9. **Visualizations:** Determine the availability of crucial services in multiple branch offices with maps and business views. With OpManager, you can easily monitor remote locations visually, and get alerted in real time before network services are disrupted.

10. **Multi-level thresholds:**OpManager offers multi-level thresholds with color codes, so you can identify show- stopping network faults and promptly take action.

Result:

The modules are executed with the commands of network monitoring tools.

EX.NO: 10(a)	FIREWALL
DATE:	

Aim:

To Knows the Uses Of Firewall And Their Uses In Windows.

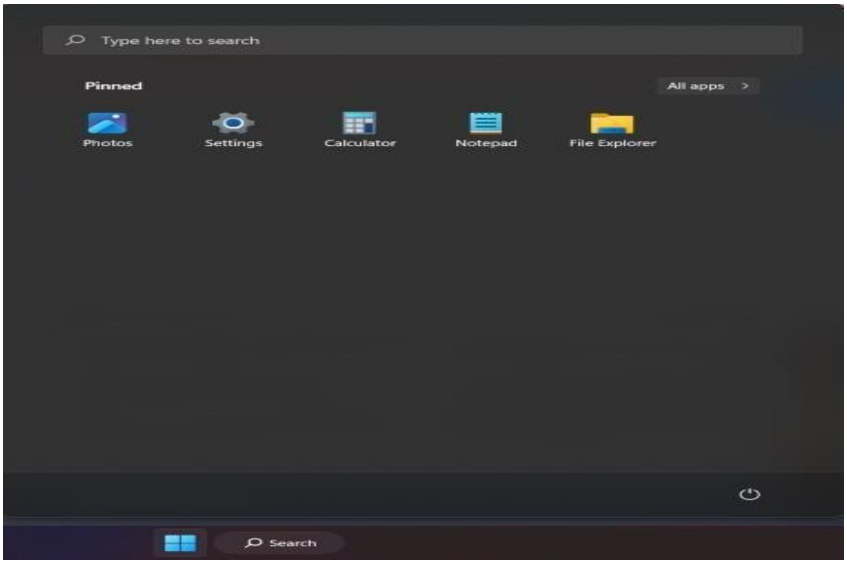
Introduction:

A firewall is a network security device, either hardware or software-based, which monitors all incoming and outgoing traffic and based on a defined set of security rules it accepts, rejects or drops that specific traffic. Please refer to the article Introduction of Firewall in Computer Network for more details. We can configure 2 types of firewalls on Windows on the basis of firewall provider:

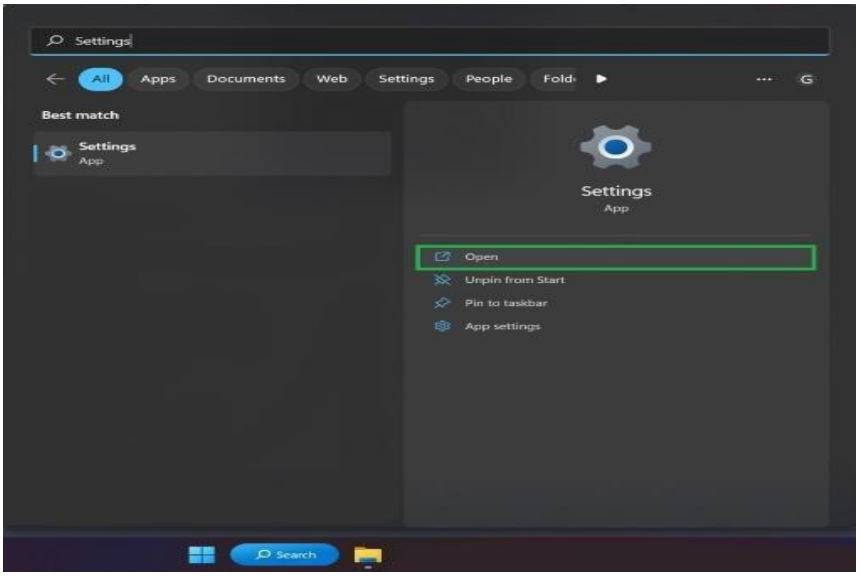
- ✓ Windows Defender (Default firewall).
- ✓ Third-party firewalls.

Configuring Firewall Defender on Windows:

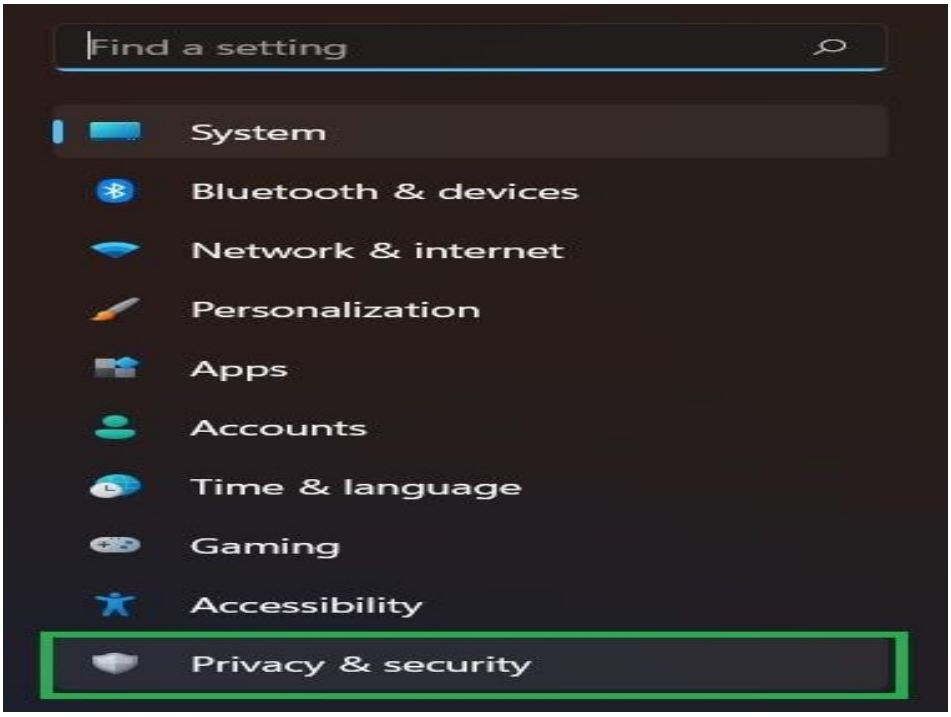
Step 1: Launch **Start** from the taskbar.



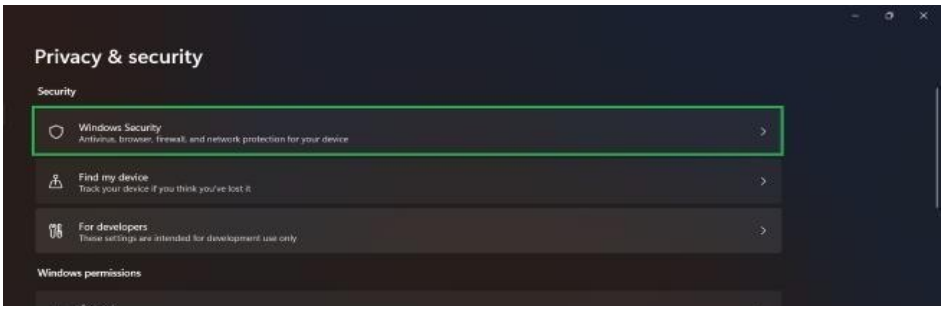
Step 2: Search **"Settings"** in the search bar if you do not find the Settings icon in Start menu.



Step 3: In the left pane of Settings, click **Privacy & security**.



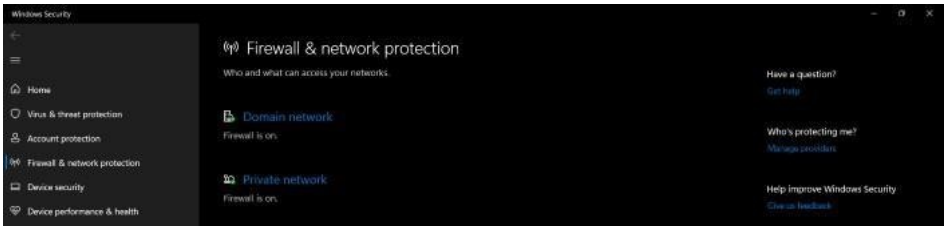
Step 4: Click **Windows Security** option in Privacy & security menu.



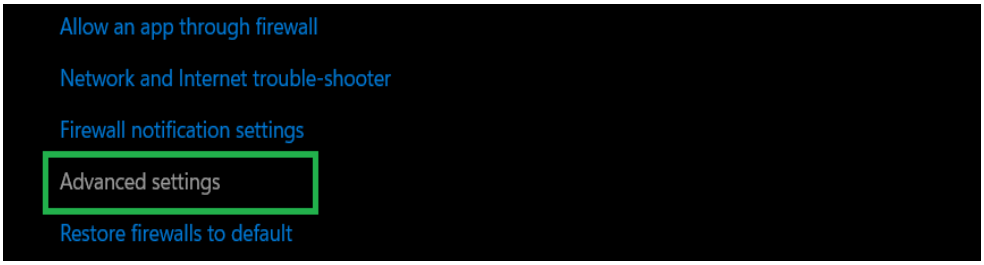
Step 5: Select **Firewall & network protection**.



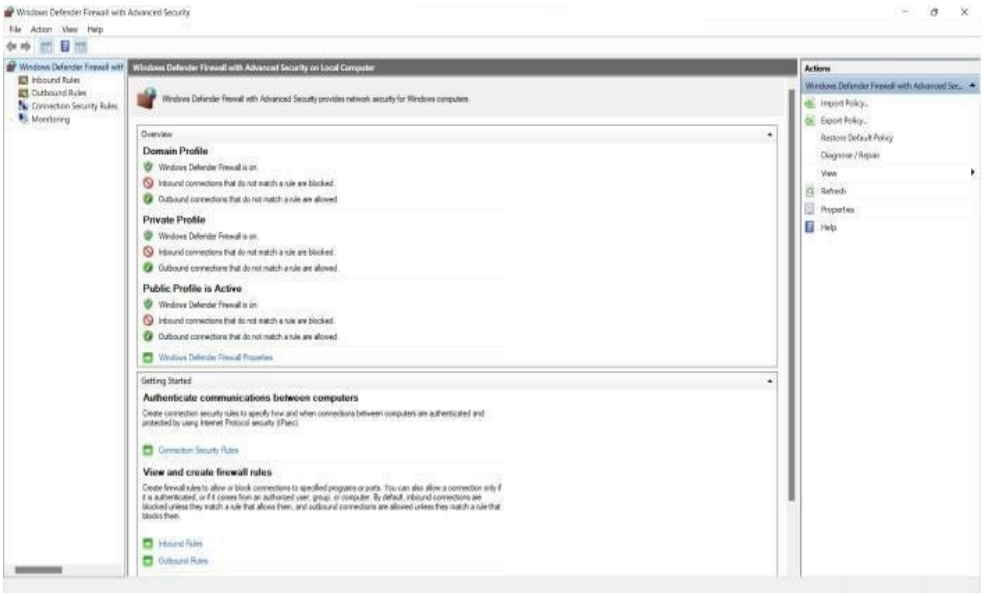
Step 6: Now **Window's Security** window will pop up windows. Here you can verify whether your Defender firewall is active or not.



Step 7: Now to configure the firewall according to your requirement, click **Advanced settings**. You will be prompted by User Account Control to give Administrative access to Windows Defender to make changes.
Click **Yes** to proceed.



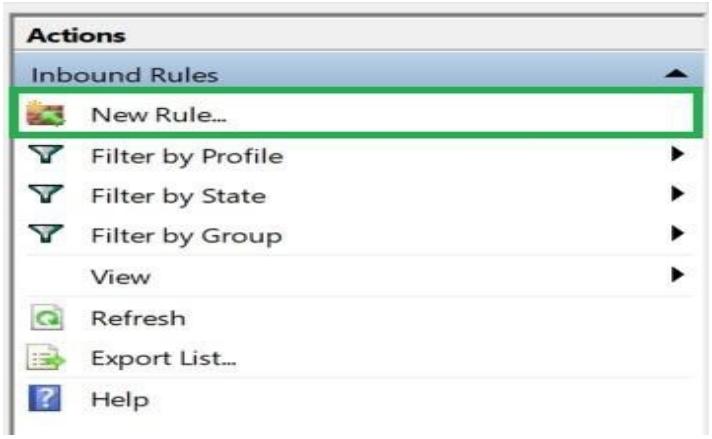
Step 8: Windows Defender Firewall with Advanced Security window will launch after giving administrative permission.



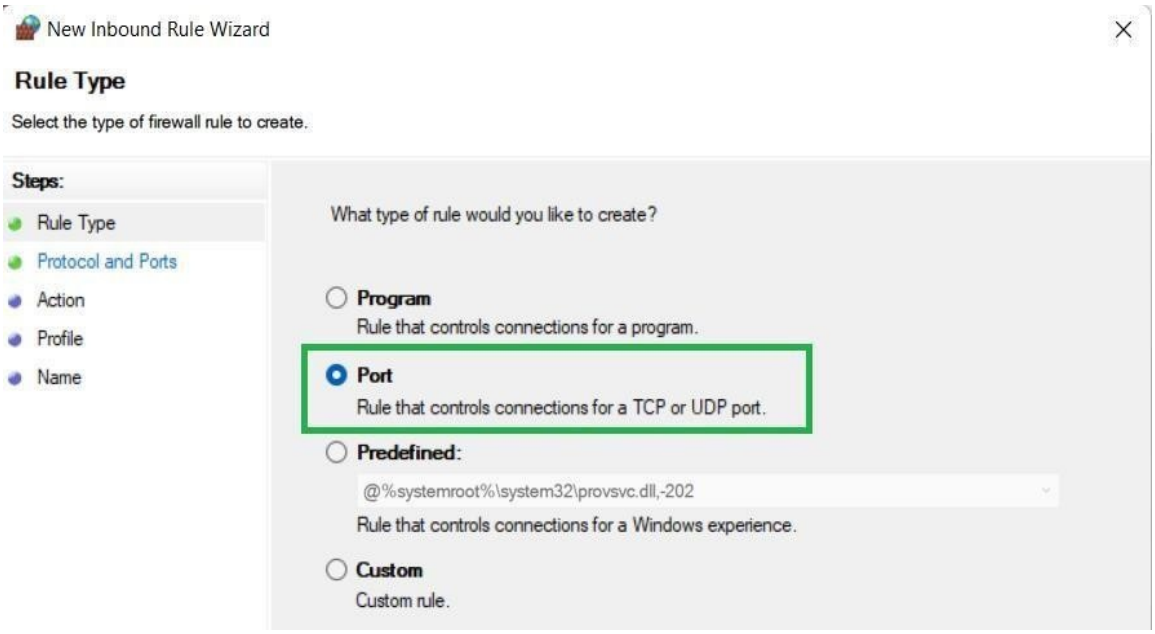
Step 9: The left pane has several options:

- Inbound rules: Programs, processes, ports can be allowed or denied the incoming transmission of data within this inbound rules.
- Outbound rules: Here we can specify whether data can be sent outwards by that program, process, or port.

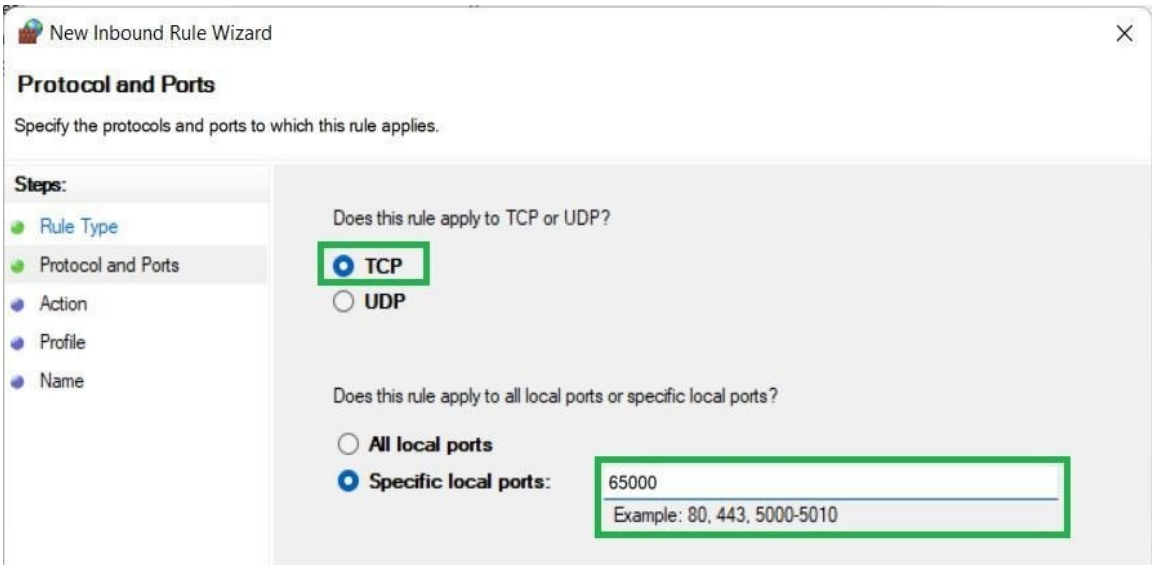
Step 10: To add a new inbound rule, select **Inbound Rules** option, then click **New Rule…** from the right pane.



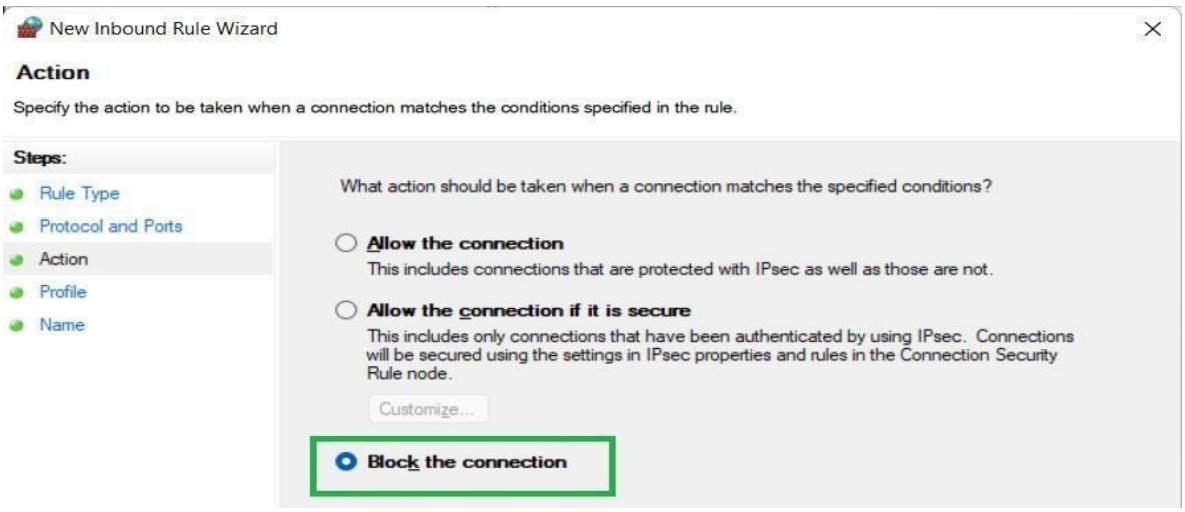
Step 11: Now we will configure an inbound rule for a network port. A **New Inbound Rule Wizard** window pops-up, select Port option and click next.



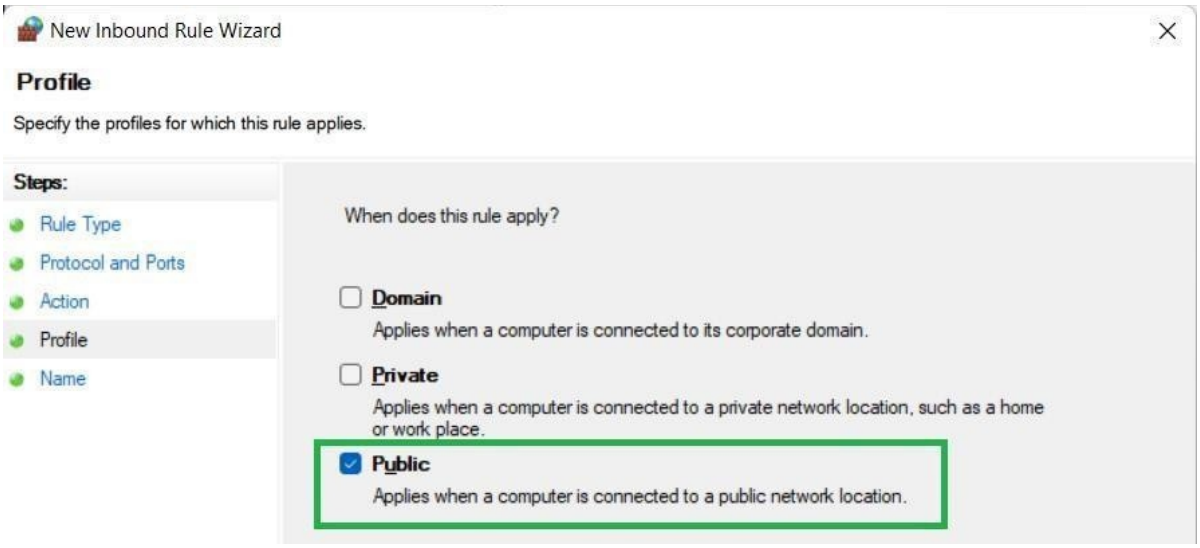
Step 12: Now select **TCP** and specify port number **65000**.



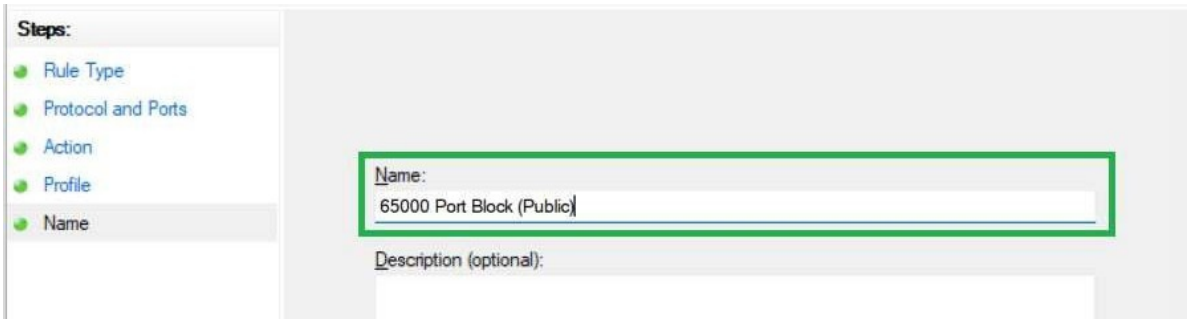
Step 13: Now we can select the action we need to take on this port. We will block the inbound connection by selecting Block the connection option then click Next.



Step 14: Here we can specify when should this rule come into action. We will keep only **Public** option selected and move **Next**.



Step 15: This is the last step. Here we provide a name to this rule so that we can keep track of it later in the Inbound rules list. Write the name "**65000 Port Block (Public)**". Click **Finish**.



Step 16: The inbound rule is successfully created. We can find "**65000 Port Block (Public)**" in the Inbound rules list.



Step 17: Right-click the rule we just created and there are multiple options with which it can be **Disabled** or **Deleted**.

Firewall can be configured on Windows in the above-mentioned way.

Result:

We protect our system with the properties of firewall in windows.

EX.NO: 10(b)	VPN
DATE:	

Aim:

To knows the uses of VPN and functionality with application.

Procedure:

- Step 1: Line up key VPN components. ...
- Step 2: Prep devices. ...
- Step 3: Download and install VPN clients. ...
- Step 4: Find a setup tutorial. ...
- Step 5: Log in to the VPN. ...
- Step 6: Choose VPN protocols. ...
- Step 7: Troubleshoot. ...
- Step 8: Fine-tune the connection.

Configuring a Virtual Private Network (VPN) involves setting up a secure and encrypted connection between your device and a remote server. VPNs are commonly used to enhance online privacy, security, and anonymity. Below, I'll provide a general overview of how to configure a VPN:

1. **Choose a VPN Service:** Start by selecting a reputable VPN service provider. There are many options available, such as NordVPN, ExpressVPN, CyberGhost, and many more. Subscribe to the VPN service and follow their setup instructions.

2. **Download and Install the VPN Client:** Most VPN services offer dedicated apps for various platforms, including Windows, macOS, iOS, Android, and Linux. Download the appropriate VPN client for your device, and install it.

3. **Launch the VPN Client:** Open the VPN client on your device. You may need to log in using your VPN service credentials.

4. **Connect to a VPN Server:** In the VPN client, choose a server location to which you want to connect. VPN services typically offer servers in various countries. Your choice of server location may affect your internet speed and the websites or services you can access.

5. **Optional Configuration Settings:** VPN clients often provide advanced settings that you can configure to meet your specific needs. These settings might include the VPN protocol (e.g., OpenVPN, L2TP, or IKEv2), a kill switch to stop internet traffic if the VPN connection drops, split tunneling to specify which apps or websites use the VPN, etc. Configure these settings according to your requirements.

6. **Connect to the VPN:** Click or tap the "Connect" or "Start" button in the VPN client to establish a connection to the selected server. Once the connection is established, your internet traffic will be encrypted and routed through the VPN server.

7. **Verify Your Connection:** You can verify that your VPN connection is active by visiting a website like WhatIsMyIP" to confirm that your IP address is now associated with the VPN server location.

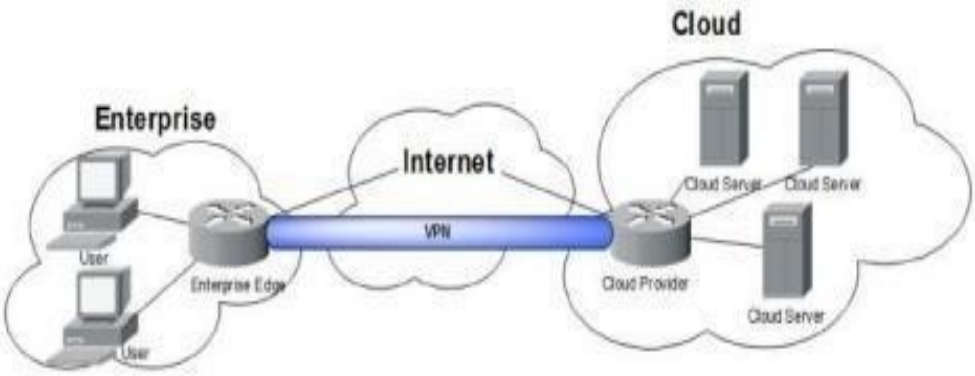
8. **Use the Internet Securely:** Your internet activity is now encrypted and secure. You can access blocked content, enhance your privacy, and protect your data from eavesdropping on public Wi-Fi networks.

9. **Disconnect from the VPN:** When you're finished using the VPN, disconnect from the server by using the VPN client.

10. **Customize Your Experience:** Some VPN services offer additional features, such as ad-blocking, anti-malware, and more. Explore your VPN service's features to see how you can tailor your VPN experience.

It's essential to choose a trusted VPN service, as they have access to your internet traffic. Be sure to read their privacy policy and terms of service to understand their data handling practices. Additionally, keep your VPN software and device operating system up to date to ensure the highest level of security.

Architecture of VPN



Surfshark VPN Options and Features:

CleanWeb:

CleanWeb was able to block ads that managed to slip even past our regular ad blockers.

MultiHop:

The MultiHop option provides some additional security but does have a negative impact on your internet speeds. After all, your traffic has to go through two VPN servers instead of one.

Whitelister:

The Whitelister for apps and websites allows you to connect to specific apps and websites you trust using your own IP address instead of that of the VPN server (split tunneling).



Surfshark Alert

Surfshark Alert is an option you can use that will notify you when your email or passwords might be in danger of being compromised.



Surfshark Search

Surfshark Search is Surfshark's answer to the anonymous search engine DuckDuckGo. However, both Surfshark Alert and Surfshark Search are paid expansions of the normal Surfshark subscription.

Surfshark VPN for torrenting

Surfshark has specialized P2P servers that help you download torrents securely. You don't have to manually select these servers: they are automatically activated when you open a torrenting program.

Camouflage Mode

Camouflage Mode is Surfshark's version of "obfuscation technology." It's primarily designed to disguise your VPN traffic to bypass any content filtering and make it more difficult to monitor your activities. It runs automatically when you pick OpenVPN (TCP or UDP) as its protocol.

NoBorders Mode

No Borders Mode is a mode in which you can use Surfshark and access the internet freely even in restrictive regions. When Surfshark detects any kind of restrictions on your network, it automatically enables the NoBorders mode. This gives you a selected list of servers that perform well despite network restrictions.

GPS spoofing

Recently, Surfshark has added a GPS spoofing function to its service. GPS spoofing allows you to change your virtual location, even when it isn't based on your IP address. Surfshark's GPS spoofing option is only available on Android for now.

Kill Switch

A kill switch is an essential security feature for any VPN. It instantly "kills" your internet connection if your VPN stops working for whatever reason. This prevents data leaks (like your IP address) and keeps you protected when your VPN falters. Surfshark offers a kill switch on Windows, macOS, iOS, and Android. You can turn it on by going into the settings in the Surfshark app.

Logging and privacy

Surfshark has a "no-logs" policy. They have made every effort to require as little personal information from you as possible for your VPN to work. Secondly, Surfshark — like most VPNs — requires certain information to monitor their service. For example, they use anonymized information to keep track of how busy their servers are and to see if there are any connection issues. As mentioned previously, they have a strict "no-logs" policy, and we couldn't find anything in their regulations that contradict this.

COCLUSION

Furthermore, Surfshark recently switched to a RAM-only server network, which means that there is no longer any data on physical servers. This form of storage also ensures that data is automatically deleted when you disconnect. Your data will therefore only be stored temporarily and cannot be retrieved afterward. In addition, Surfshark is one of the first VPNs to offer two-factor authentication (2FA). This means logging in happens through two steps, which is an effective way to protect yourself against all kinds of online attacks. For Surfshark Alert users, this two-step authentication is mandatory, while the 'normal' Surfshark user can choose to use it for extra protection.